arXiv:2611.08713v1 [cs.DC] 5 Nov 2026

RESSERVE: ACCURATE AND EFFICIENT FP4 LLM INFERENCE WITH RESIDUE COMPENSATION

Shuo Rong[1] **Ruibo Fan**[1] **Jiawen Wen**[1] **Xiaowen Chu**[1,2]

[1]The Hong Kong University of Science and Technology (Guangzhou), Guangzhou, China
[2]The Hong Kong University of Science and Technology, Hong Kong SAR, China

{srongaa, ruibo.fan, jwen341}@connect.hkust-gz.edu.cn
xwchu@hkust-gz.edu.cn

ABSTRACT

Block-scaled 4-bit floating-point (FP4) quantization reduces large language model inference costs but can degrade task quality. Higher-precision computation can mitigate this loss at the expense of some of the efficiency gains from FP4 inference. Narrow matrix computations common in decoding often leave spare hardware capacity for low-cost precision recovery. ResServe combines an FP4 activation base with an FP4-quantized residue representing information lost during quantization. It uses this spare capacity to compute the residue's contribution and combines results on-chip, avoiding extra result transfers. We apply this principle to linear layers and decode attention, while selective compensation limits added work when processing input prompts. On NVIDIA B200, compensated linear kernels using the NVFP4 format improve numerical precision while retaining latency close to uncompensated execution for the evaluated operations with few rows. In the evaluated NVFP4 inference configurations, ResServe adds at most 3.4% request-latency overhead over W4A4 and reduces request latency by 22.4–53.3% relative to W4A8. Across five models, code-generation accuracy improves over same-format uncompensated inference, with mean relative gains of 15.04% for NVFP4 and 53.57% for MXFP4.

Our code is available at github.com/ShawRong/resserve.

1 INTRODUCTION

Large language models (LLMs) support coding assistants (Chen et al., 2021), agentic tool use (Schick et al., 2023), and multi-turn chat. Reducing inference costs while preserving model quality is important for these applications. Quantization reduces these costs by lowering the precision of model weights and activations, the intermediate values produced during computation (Frantar et al., 2023; Lin et al., 2024; Xiao et al., 2023). Recent GPUs natively support four-bit floating-point (FP4) computation (NVIDIA Corporation, 2026a), offering lower storage requirements and lower latency. However, limited precision introduces quantization error that can degrade model quality (Egiazarian et al., 2026; Cook et al., 2025).

This matters especially for complex reasoning tasks. Models such as DeepSeek-R1 and Qwen3 generate reasoning traces to solve mathematical and coding problems (Guo et al., 2025; Yang et al., 2025). Low-bit quantization can reduce reasoning accuracy (Liu et al., 2025a). Under some quantization settings, models also generate longer chains of thought while producing less accurate answers (Lotfi et al., 2026). Generating more tokens can offset some of the inference speedup from quantization. Precision recovery may therefore affect both answer quality and the computation needed to produce an answer.

Prior work mitigates low-bit quantization error through improved quantization algorithms and transformations of weights and activations (Frantar et al., 2023; Lin et al., 2024; Dettmers et al., 2024; Kim et al., 2024; Xiao et al., 2023; Shao et al., 2024; Ashkboos et al., 2024; Liu et al., 2025b; Sun et al., 2025; Egiazarian et al., 2026). Unlike weights, which can be quantized offline, activations must be quantized during inference. Online quantization and associated transformations therefore require efficient execution (Xiao et al., 2023; Ashkboos et al., 2024; Egiazarian et al., 2026). Another approach pairs four-bit weights with eight-bit activations (Lin et al., 2025; Zheng et al., 2026; Zhang et al., 2024; Baek et al., 2025). These configurations can improve activation precision, but their execution paths do not retain all the computational advantages of native FP4 (Jarmusch et al., 2025; NVIDIA Corporation, 2026a). This raises the question: *how can we recover precision in FP4 inference while retaining its execution efficiency?*

Our observation is that additional computation does not proportionally increase latency. During decoding, which generates tokens one step at a time, small batches limit the number of token rows processed by linear layers. Decode attention processes few queries for the current token against keys accumulated over the context. The resulting matrix computations are narrow and may leave GPU resources underused. This spare capacity offers room for additional precision-recovery computation. Exploiting it requires suitable execution layouts and low-cost handling of intermediate results.

ResServe uses this spare capacity to compute a correction for activation quantization error. It quantizes the difference between the original activation and its FP4 approximation into FP4. ResServe computes both the base output and a correction from the quantized residue, combining them to recover precision. We call this process residue compensation. ResServe adapts how compensation is computed to the matrix dimensions and combines base and compensation results on-chip, avoiding separate writes to GPU memory followed by an addition. During decoding, it applies this principle to linear layers and attention. Prefill processes the input prompt and typically handles more tokens at once, making full compensation more costly. ResServe compensates a subset of activation channels during prefill to control the added computation, a strategy we call selective prefill compensation.

We implement ResServe in vLLM (Kwon et al., 2023), integrating compensation for linear layers and decode attention. The system supports NVFP4 and MXFP4 for linear computation and uses an NVFP4 KV cache for attention. Relative to same-format uncompensated inference, code-generation accuracy improves by an average of 15.04% for NVFP4 and 53.57% for MXFP4 across five models. With MXFP4, mean relative accuracy gains on the HMMT and AIME mathematics benchmarks (Dekoninck et al., 2026) are 50.23% and 33.34%, respectively. On the same code-generation benchmark, total output tokens decrease by an average of 3.18% for NVFP4 and 9.16% for MXFP4.

ResServe retains near-W4A4 request latency in the evaluated NVFP4 configurations with selective prefill disabled, while reducing latency relative to W4A8 by 22.4% on Qwen3-8B and 53.3% on Qwen3-32B. With the prefill compensation ratio of 0.25 used in the quality evaluation, MXFP4 ResServe adds 7.0–9.4% request-latency overhead over same-format W4A4.

This paper makes the following contributions:

- We identify spare hardware capacity in narrow decoding operators that can accommodate additional low-precision computation for precision recovery without a proportional latency increase.
- We design execution layouts that use this spare capacity for residue compensation in linear layers and decode attention. On-chip result combination and selective prefill compensation control the added execution cost.
- We build a complete inference system with NVFP4/MXFP4 linear computation and NVFP4 KV caching. The system improves mean coding and mathematics accuracy across five models, while low-load NVFP4 serving measurements show low integration overhead.

2 Background

Block-scaled FP4 represents tensor elements with four-bit floating-point values and scales shared within groups. NVFP4 and microscaling FP4 (MXFP4) both use E2M1 values but differ in scaling. NVFP4 shares an FP8 E4M3 scale within each 16-element block and an additional 32-bit floating-point (FP32) scale across the tensor (NVIDIA et al., 2025; Cook et al., 2025). MXFP4 shares an E8M0 power-of-two scale across 32 consecutive elements, without an additional tensor-level scale (Rouhani et al., 2023). Figure 1 shows both representations and their linear-layer execution flow.

Here, W4A4 denotes block-scaled FP4 weights and activations. W4A8 denotes block-scaled FP4 weights paired with FP8 or block-scaled MXFP8 activations.

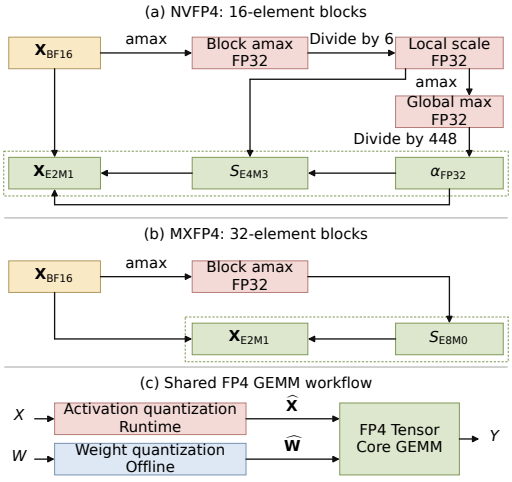

Figure 1. NVFP4 and MXFP4 representations and their W4A4 linear-layer execution. Activations are quantized at runtime and weights offline.

We use QDQ to denote quantization followed by dequantization. For an input x, we write the resulting value as $\hat{x} = \mathrm{QDQ}(x)$. The quantization residue is the difference

from the original value, $r = x - \hat{x}$. The same notation applies to activation, weight, and quantized residue matrices, denoting the numerical values represented by their FP4 encodings rather than the codes themselves.

We write a linear projection as $\mathbf{X}^{M\times K}\mathbf{W}^{K\times N} \rightarrow \mathbf{Y}^{M\times N}$, where M is the actual number of token rows in the current GEMM. During standard autoregressive decoding, each active request contributes one token row per step. The dimensions K and N denote the input and output channels, respectively. An input channel is a column of $\mathbf{X}$ and the corresponding row of $\mathbf{W}$.

3 Motivation

Precision recovery must account for both quantization error and execution cost. Plain block-scaled FP4 can leave a task-quality gap relative to BF16 (Table 2). Wider activations can reduce error but sacrifice native FP4's theoretical throughput advantage. Residue compensation recovers discarded activation information through additional FP4 computation.

Compensation latency depends on how this additional work executes. A separate compensation GEMM adds a kernel launch and output traffic for storing and adding the correction. Decode linear layers and attention have narrow token and query dimensions, respectively, which can leave tile capacity underused. When the original and extended computations fit within the same tile, residue computation can occupy otherwise idle positions. This leaves the tile count along that dimension unchanged. Exploiting this capacity requires organizing the compensation computation and merging its results on-chip. Prefill processes more tokens at once, so compensating only a subset of channels helps control the added computation.

4 ResServe Design

4.1 Overview

ResServe uses block-scaled FP4 for attention projections and feed-forward linear layers (Figure 2). ResAttn performs decode attention over the existing paged NVFP4 KV cache. For models with hybrid attention, Gated DeltaNet retains BF16 core computation and FP32 recurrent state.

ResServe recovers information lost during activation quantization through additional low-precision computation (Figure 3). It computes with an FP4 base and an FP4-quantized residue, combining their contributions on-chip. This avoids a separate correction tensor in global memory and an extra kernel to add it.

ResServe adapts compensation to the matrix dimensions

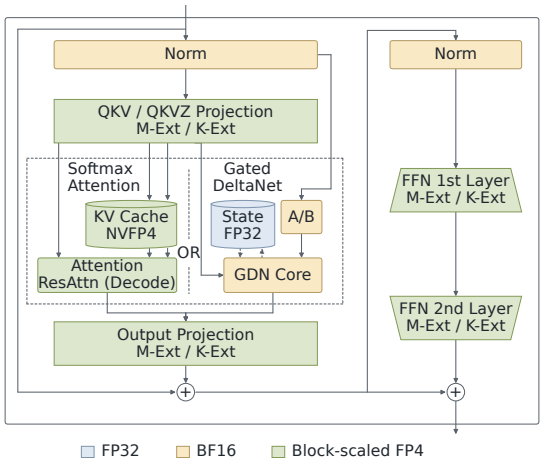

Figure 2. Precision assignment in ResServe. Colors indicate the numerical precision used by each model component.

and to where the base and compensation results must be combined. For skinny operators common in decoding, M-Ext and ResAttn extend computation along the narrow dimension to use otherwise idle execution capacity. M-Ext arranges the base and residue as paired activation rows in linear layers, then adds their corresponding outputs. ResAttn applies the same principle to multiplication of queries and keys, pairing the query base and residue and combining their scores before softmax while preserving the existing NVFP4 KV-cache layout. For prefill with larger row counts, ResServe selectively compensates residue channels to control the added computation (Section 5.3).

4.2 On-Chip Residue Compensation

ResServe adds an activation residue term to the base low-precision linear output:

$$\mathbf{Y} \approx \hat{\mathbf{X}}\hat{\mathbf{W}} + \hat{\mathbf{R}}_{\mathcal{S}}\hat{\mathbf{W}}_{\mathcal{S}}, \tag{1}$$

where the first term is the plain block-scaled FP4 GEMM and the second recovers information lost to activation quantization. The residue is $\mathbf{R} = \mathbf{X} - \hat{\mathbf{X}}$, and $\mathcal{S}$ specifies the compensated channels. The correction uses FP4-quantized values of these residue channels and their corresponding weight rows. Full compensation includes all channels and reuses the base weight. Selective compensation includes only a subset to reduce the added computation. ResServe uses this option for prefill with more token rows; Section 5.3 describes channel selection and execution.

4.3 Full-Residue Decode Compensation (M-Ext)

M-Ext organizes full-residue compensation as a GEMM extended along the token dimension. The base activations

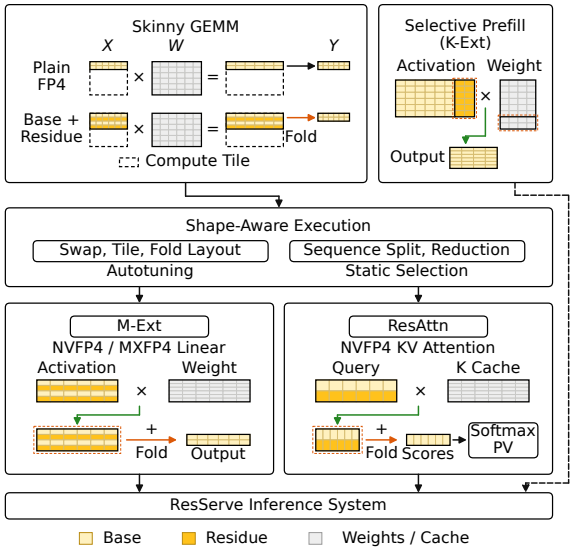

Figure 3. ResServe overview. X, W, and Y denote activations, weights, and outputs.

and residues multiply the same weight, so M-Ext stacks them as paired rows and adds the corresponding outputs:

$$\mathbf{Z} = \begin{bmatrix} \hat{\mathbf{X}} \\ \hat{\mathbf{R}} \end{bmatrix} \hat{\mathbf{W}} = \begin{bmatrix} \hat{\mathbf{X}}\hat{\mathbf{W}} \\ \hat{\mathbf{R}}\hat{\mathbf{W}} \end{bmatrix}, \qquad \mathbf{Y}_{j,:} = \mathbf{Z}_{j,:} + \mathbf{Z}_{M+j,:}. \tag{2}$$

We call the elementwise addition of paired base and residue output rows Fold. It restores the original $M \times N$ output shape. This organization doubles the logical row count from M to $2M$ without extending the reduction dimension K or duplicating the weight. For skinny GEMMs in decoding, the added rows can use otherwise idle execution capacity. Section 5.1 develops the layouts and execution.

4.4 Residue-Compensated Decode Attention (ResAttn)

The multiplication of queries and keys in decode attention is also a skinny GEMM. In grouped-query attention (Ainslie et al., 2023), a small number of queries within each group sharing one KV head is multiplied against keys from the cached sequence. ResAttn applies the same compensation principle by splitting the BF16 query into an FP4 base and a quantized residue.

Unlike the linear output fold, attention must combine the compensated scores before softmax. For the query-head group sharing one KV head, let $\hat{\mathbf{Q}}$ and $\hat{\mathbf{R}}$ denote the FP4 base and quantized residue of the BF16 queries. Each row corresponds to one query head. The logical computation is

$$\mathbf{S} \approx \frac{\hat{\mathbf{Q}}\hat{\mathbf{K}}^T + \hat{\mathbf{R}}\hat{\mathbf{K}}^T}{\sqrt{d}}, \quad \mathbf{O} = \mathrm{softmax}(\mathbf{S})\hat{\mathbf{V}}. \tag{3}$$

Here d is the head dimension.

ResAttn preserves the existing paged NVFP4 KV-cache layout. Section 5.2 explains the matrix multiplications under this layout, together with sequence splitting and reduction selection.

5 Efficient Execution and System Integration

ResServe fuses residue generation and quantization into the activation quantization pass. The quantizer forms residues from the input and reconstructed base values in registers, avoiding a separate pass to read the base operand. It directly emits the layout consumed by the FP4 GEMM, eliminating a separate activation reordering kernel. Appendix B.2 details the NVFP4 scale computation and fused quantization procedure.

5.1 M-Ext Kernel: Tiling and Output Folding

M-Ext combines operand swapping, tile selection, and output layouts to use spare execution capacity and implement Fold in Eq. 2 efficiently. The layouts place each base output and its corresponding residue output in the same thread's registers, where they are added.

Swapping the operands maps tokens to the narrower dimension of the matrix multiply-accumulate (MMA) output tile. A conventional linear layer evaluates $\hat{\mathbf{X}}^{M\times K}\hat{\mathbf{W}}^{K\times N}$, producing N output features (channels) for each of M tokens. In the NVFP4 kernels used here, MMA output tiles have a larger minimum row count than column count (NVIDIA Corporation, 2026a; Ye et al., 2025). The conventional layout assigns tokens to output rows, so small decode batches leave many tile rows unused.

M-Ext applies this transformation to its stacked base–residue operand $\hat{\mathbf{A}} = [\hat{\mathbf{X}}; \hat{\mathbf{R}}] \in \mathbb{R}^{2M\times K}$:

$$\hat{\mathbf{W}}^{T,N\times K}\hat{\mathbf{A}}^{T,K\times 2M} = \mathbf{Z}^{T,N\times 2M}. \tag{4}$$

After the swap, rows of the result tile correspond to output channels, and columns correspond to the base and residue rows. For small decode batches, the kernel can therefore use tiles with fewer columns. Let B denote the number of activation rows processed by the GEMM, including residue rows when compensation is enabled. Plain NVFP4 has $B = M$, whereas M-Ext has $B = 2M$.

Figure 6a compares both orientations for plain NVFP4 GEMM with $N = K = 4096$ and a fixed 128×64 tile.

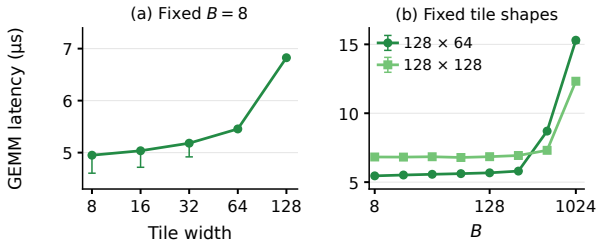

Figure 4. Pure NVFP4 GEMM latency. (a) Varying tile width at fixed $B = 8$. (b) Varying B with two fixed tile shapes.

Swapping gives 1.04–1.10× speedups for $M = 1$–32, while the two orientations are nearly equal at $M = 64$.

Residue compensation can use unused space within tiles and idle GPU resources to keep latency overhead low. After the operand swap, result-tile columns correspond to tokens; we refer to the tile size along this dimension as tile width. Figure 4 examines how tile width and input size affect pure NVFP4 GEMM latency. When the base and residue rows together fit within the tile width, compensation uses otherwise unused positions without increasing the number of tiles. If the extension exceeds that capacity, a wider tile is another option. Figure 4a fixes $B = 8$ and increases tile width from 8 to 64; GEMM latency rises from 4.95 to 5.46 μs, or 10.2%. Enlarging tiles to accommodate residues may therefore limit tile-count growth at a modest latency cost.

When the extension requires more tiles, the added tasks may still use parallel capacity left idle by the original workload. Figure 4b fixes the input and output channel counts at $K = N = 4096$ and varies B, giving a transposed result with 4096 rows and B columns. For output-channel tile size T_N and tile width T_B, the logical tile count is

$$n_{\text{tiles}} = \left\lceil \frac{N}{T_N} \right\rceil \left\lceil \frac{B}{T_B} \right\rceil . \tag{5}$$

Each tile covers 128 output channels and 64 token positions, dividing the output-channel dimension into 32 tiles. Increasing B from 128 to 256 raises the number of tiles along the token dimension from 2 to 4. The total tile count thus grows from 64 to 128, both below the GPU's 148 SMs. Measured latency rises by only 2.2%, from 5.68 to 5.80 μs.

As the input grows further, wider tiles can reduce the number of tasks to schedule. At $B = 512$, the preceding configuration produces 256 tiles. Increasing tile width to 128 reduces this count to 128 and latency from 8.70 to 7.31 μs. This is consistent with executing the tiles in fewer waves. Narrower tiles expose more independent tasks, whereas wider tiles fit more base and residue outputs and limit the increase in tile count.

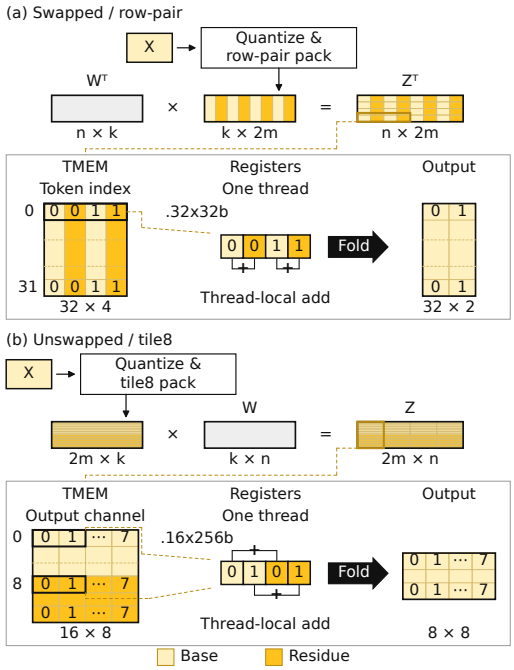

Figure 5. M-Ext packing and output folding. Swapped row-pair and unswapped tile8 layouts match TMEM loads to thread-local base–residue pairs. Insets show representative fragments.

Matching activation packing to TMEM loads keeps each base–residue pair in the same thread's registers. Figure 5 shows the paired layouts. On B200, Tensor Cores accumulate into Tensor Memory (TMEM), and the epilogue loads those accumulators into registers (NVIDIA, n.d.b). The TMEM load shape determines which accumulator values each thread receives. Each thread adds the pair in FP32 before the output cast.

In the swapped layout, the quantizer interleaves each base row with its residue row. Their outputs occupy adjacent columns in the accumulator tile in TMEM. A `.32x32b` TMEM load brings the corresponding values into thread-local registers for Fold. For shapes better served by unswapped tiles, the quantizer packs eight base rows followed by their eight residue rows. A `.16x256b` TMEM load places each pair in the same thread's registers.

After Fold, the epilogue writes the combined output to global memory, either through shared memory using a Tensor Memory Accelerator (TMA) store or directly from registers. Figure 6b compares on-chip Fold against writing the expanded BF16 output and combining each pair with a separate Fold kernel. With identical prepared operands, $N = K = 4096$, and fixed 128 × 64 tiles, on-chip Fold gives 1.34–1.41× speedups for $M = 1$–64.

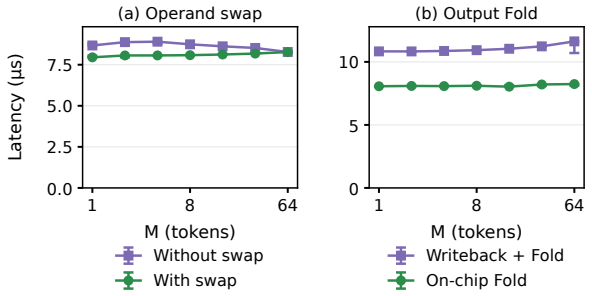

Figure 6. NVFP4 execution ablations. (a) GEMM with and without operand swapping. (b) On-chip Fold versus a separate Fold kernel. Timing excludes quantization; bars show min–max across three processes.

M-Ext jointly autotunes tile shapes, activation packing, TMEM loads, and output stores. For each combination of M, N, and K, it measures compatible configurations and selects the fastest, including activation packing, GEMM, and Fold in the timing.

5.2 ResAttn Execution

ResAttn applies M-Ext's paired computation to decode attention and combines the compensated scores before softmax. Its execution must preserve the existing KV-cache layout and expose parallelism for long sequences.

ResAttn pairs the query base and residue computations and folds their scores before softmax (Figure 3). A separate query quantization kernel generates and packs FP4 base and residue rows from BF16 queries before the attention kernel runs. The attention kernel directly uses the cached FP4 keys in the product of queries and keys (QK). As in M-Ext, ResAttn swaps the operands so that query base and residue outputs occupy columns of the score tile. It adds the corresponding scores on-chip in FP32 before applying online softmax (Dao et al., 2022).

ResAttn preserves the paged NVFP4 layout of both K and V. Both operands retain scale groups along the head dimension. This grouping matches the reduction dimension of QK, allowing direct FP4 execution. In the product of probabilities and values (PV), however, the reduction dimension follows sequence positions rather than the head dimension. Direct block-scaled FP4 PV would require grouping V scales along that dimension instead. To retain the existing cache layout, ResAttn converts V from NVFP4 to FP8 on-chip. ResAttn computes PV with FP8 operands, using FP32 accumulation and normalization.

ResAttn adopts KV sequence splitting to expose more parallel work. This is useful when the batch and KV-head dimensions provide too few thread blocks (Dao et al., 2023; Ye et al., 2025). Each split computes local attention outputs and softmax statistics; reduction combines these outputs using the statistics to preserve normalization across the full sequence. More splits can improve parallelism, but also add storage for partial outputs, synchronization, and reduction work. ResAttn jointly selects the split count and reduction method to balance these costs (Figure 3). Preset rules use batch size, KV-head count, sequence length, and the GPU's SM count to select the configuration without a timing search during serving.

5.3 Selective Prefill Execution

Selective prefill controls added computation by compensating only a subset of channels. Prefill processes more tokens concurrently, making the computational overhead of full compensation more pronounced. Prior work shows that channels differ in quantization sensitivity and that activation statistics help identify channels worth protecting (Lin et al., 2024; Zhao et al., 2024). ResServe uses block-local selection for block-scaled FP4, with implementations for both NVFP4 and MXFP4. Calibration uses mean absolute activation as a heuristic salience score and selects a fixed number of channels within each local group. Inference reuses these selections. Our current NVFP4 and MXFP4 implementations use 8-channel selection groups for partial compensation ratios of 0.125, 0.25, and 0.5. Both also support full compensation at 1.0. M-Ext is designed for full-residue compensation in decoding, while selective compensation controls the added work in prefill.

Selected residue channels and matching weight rows can be incorporated into the original GEMM. As shown in the upper-right panel of Figure 3, we append quantized residues as activation columns and the corresponding quantized weights as weight rows:

$$\begin{aligned}\mathbf{Y} &\approx \begin{bmatrix}\hat{\mathbf{X}} & \hat{\mathbf{R}}_{\mathcal{S}}\end{bmatrix}\begin{bmatrix}\hat{\mathbf{W}} \\ \hat{\mathbf{W}}_{\mathcal{S}}\end{bmatrix} \\ &= \hat{\mathbf{X}}\hat{\mathbf{W}} + \hat{\mathbf{R}}_{\mathcal{S}}\hat{\mathbf{W}}_{\mathcal{S}}.\end{aligned} \tag{6}$$

This concatenation extends the GEMM reduction dimension, hence the name K-Ext. A single FP4 GEMM directly sums the base and compensation terms of Eq. 1. Offline, we append the selected floating-point weight rows to the original weights, then quantize using the extended block layout. Here, $\hat{\mathbf{W}}_{\mathcal{S}}$ denotes the selected weights quantized in this extended layout. At runtime, the fused quantizer generates the base and selected residues in the corresponding concatenated layout. For a selected channel fraction ρ, the effective reduction dimension is

$$K_{\text{eff}} = K(1+\rho).$$

Block-local selection with a fixed number of channels per group lets quantization directly produce the extended activation. Global selection can leave threads holding different numbers of selected channels, requiring runtime reordering to form contiguous operands (Figure 7). Even with offline selection indices, each new input still requires the corresponding indexing and data movement. ResServe selects the same number from every group, making output offsets predictable. The fused quantizer computes residues in registers from the original inputs and their FP4 base values. It extracts selected residues and computes independent scales for contiguous residue blocks. It then quantizes and packs them directly into the extension, avoiding a separate channel-reordering step. Appendix Algorithm 1 details the NVFP4 implementation.

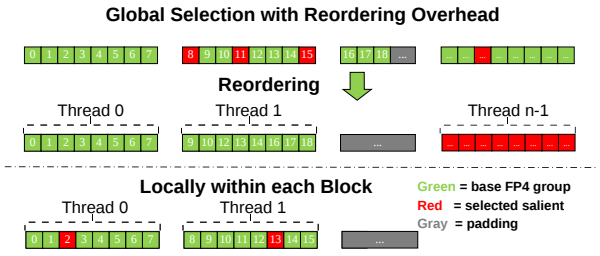

Figure 7. Channel selection under SIMT. Global selection uses runtime channel reordering (top); block-local selection with a fixed number of channels per group supports fused packing (bottom).

5.4 System Integration

To support selective prefill compensation, we extend NVIDIA TensorRT Model Optimizer (NVIDIA Corporation, 2026b) to save each layer's selected channels and execution mode during export. The vLLM loader restores this information together with the corresponding weight layout. Weight preparation is performed once during export, avoiding runtime weight reordering. Prefill and decode share the base weight. Partial compensation additionally stores the selected weight rows for the extended GEMM (Appendix D.5). An optional offline search assigns layer-wise ratios under a compensation budget (Appendix E.1).

The online runtime is built on vLLM 0.26.0. Both linear paths build on FlashInfer (Ye et al., 2025); M-Ext adds residue input layouts, tactic tuning for each input shape, and the output fold of Section 5.1. ResAttn reuses the existing KV cache layout, preserves the prefill attention path, and replaces only the decode attention computation. The baseline uses TensorRT-LLM's decode attention kernel through FlashInfer (NVIDIA, n.d.a).

6 Evaluation

6.1 Experimental Setup

We primarily evaluate on NVIDIA B200 (SM100), with additional RTX 5090 (SM120) NVFP4 linear-kernel results in Appendix A. Task evaluations cover Qwen3-4B/8B/32B, Qwen3.8-27B, and Swift-1.5-Qwen3.8-27B (Swift-27B) (Yang et al., 2025; Qwen Team, 2026; UkisAI, 2026). We compare NVFP4 and MXFP4 M-Ext with same-format plain FP4 and W4A8 implementations from FlashInfer (FI) and TensorRT-LLM (TRT). Plain denotes W4A4 execution without residue compensation. Precision comparisons use the same original weights and BF16 activations. Linear timing includes online activation quantization and GEMM/Fold. Appendix C.1 specifies backend formats and tuning settings.

6.2 Kernel Evaluation

6.2.1 *Linear Kernels*

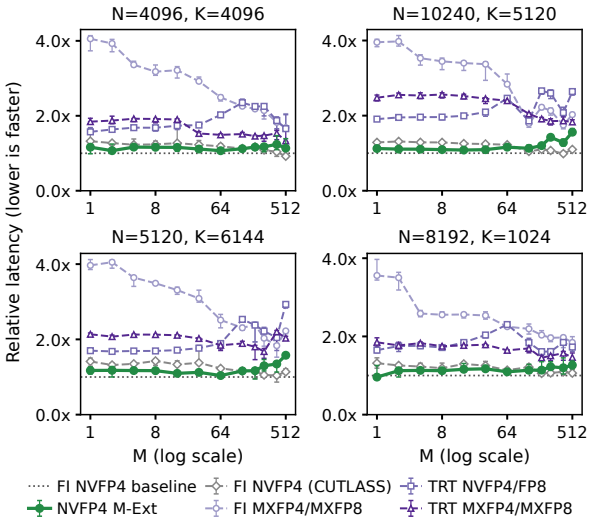

Figure 8. B200 linear latency relative to FI NVFP4 CuTeDSL (lower is better). **Green curves show our NVFP4 M-Ext.** FI NVFP4 CUTLASS is shown for reference.

For M from 1 to 64, NVFP4 M-Ext achieves median speedups of 1.53–1.83× over TRT NVFP4/FP8. Over the same row counts, MXFP4 M-Ext achieves median speedups of 1.55–1.96× over TRT MXFP4/MXFP8. NVFP4 M-Ext incurs 3.1–7.8% median overhead over plain NVFP4. The NVFP4 summaries cover 22 projection shapes; Figure 8 shows four representative shapes. Appendix Figures 18 and 19 provide the per-shape sweeps.

At full compensation, M-Ext lowers execution cost relative to K-Ext (Appendix Figure 16). With identical NVFP4 base and residue values, M-Ext is 1.47–1.48×

faster than full K-Ext for $M = 1$–128 and $N = K = 4096$. This comparison includes online activation quantization and uses independently selected kernel configurations.

M-Ext also reduces output error on Qwen3-32B against an FP32 reference (Appendix Table 3). The comparison uses 72 paired real inputs and includes both weight and activation quantization error. Pooled normalized MSE (NMSE) divides the summed squared error by the summed squared reference values across all evaluated inputs. NVFP4 M-Ext lowers NMSE by 16.6% relative to plain NVFP4 and 24.8% relative to TRT NVFP4/FP8. MXFP4 M-Ext reduces NMSE by 20.3% relative to plain MXFP4, approaching the output precision of MXFP4/MXFP8.

Increasing the prefill compensation ratio reduces NMSE in both FP4 formats on Swift-27B and Qwen3.8-27B (Figure 9(a–b)), using 192 matched prefill input groups per model. At ratio 0.5, NVFP4 NMSE is less than 1.8% above TRT NVFP4/FP8 on both models. Full compensation reduces error below same-format W4A8 on both models and formats.

Selective compensation reduces prefill overhead relative to full compensation (Figure 9(c–f)). At $M = 1024$ and 2048, median latency across 17 shapes is 1.11–1.14× plain NVFP4 at ratio 0.125, versus 1.69–1.97× for full compensation. Appendix Figure 20 provides the per-shape sweeps.

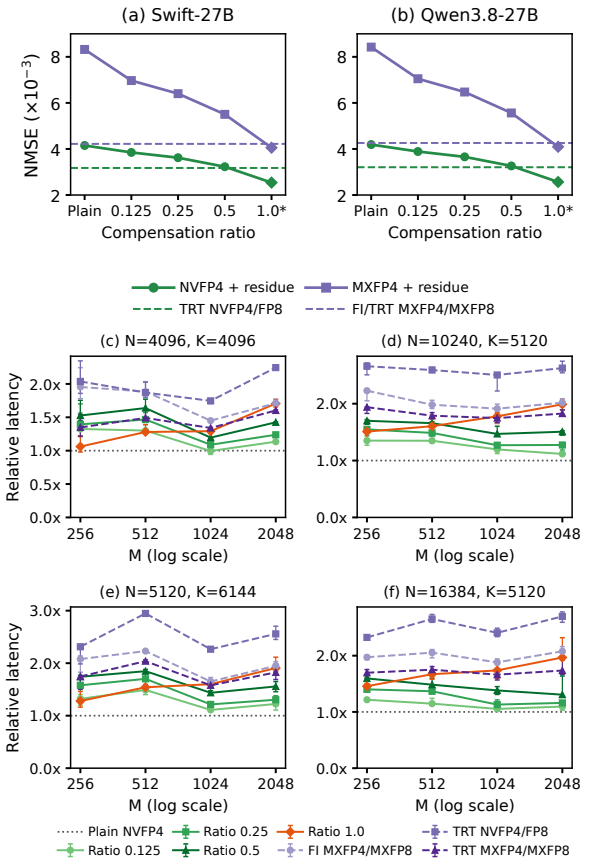

Figure 9. Selective compensation: (a–b) linear NMSE; (c–f) B200 linear latency relative to FI plain NVFP4 (lower is better). (*) marks full compensation; ratio positions in (a–b) are categorical.

6.2.2 *Decode Attention*

ResAttn achieves up to 1.40× speedup over TensorRT-LLM, with a median speedup of 1.19× across 48 configurations. It is faster in 47 configurations (Figure 10(a–c)). Both kernels use identical BF16 queries and prepared page32 NVFP4 KV, and directly produce BF16 output. The TRT baseline converts BF16 queries to FP8 before attention. Timing covers query conversion and attention, excluding cache construction and KV append.

ResAttn also lowers pooled NMSE by 66.3% on Swift-27B and 68.1% on Qwen3.8-27B across 192 paired real-input attention instances per model (Figure 10(d)). The FP32 attention reference uses the same NVFP4 KV, isolating attention computation error from KV quantization loss. Using FP32 attention on the original BF16 Q, K, and V as the reference, ResAttn reduces total NMSE relative to TRT-LLM by 14.98% on Swift-27B and 16.24% on Qwen3.8-27B.

6.3 End-to-End Latency

We evaluate whether kernel efficiency benefits carry over to full-model serving with M-Ext and ResAttn. The low-load HTTP evaluation covers NVFP4 and MXFP4 on Qwen3-8B and Qwen3-32B, with one SM100 GPU per model. Requests contain 8,192 input and 256 output tokens, with Poisson arrivals at 1 request/s. Table 1 reports time to first token (TTFT), mean time per output token after the first (TPOT), and complete request latency. The W4A8 baselines use TensorRT-LLM's NVFP4/FP8 and MXFP4/MXFP8 GEMMs for NVFP4 and MXFP4 weights, respectively.

With selective prefill compensation disabled, NVFP4 ResServe adds 3.4% request-latency overhead on Qwen3-8B and retains nearly unchanged latency on Qwen3-32B relative to W4A4. MXFP4 ResServe uses the quality evaluation's prefill compensation ratio of 0.25, adding 7.0% and 9.4% overhead over same-format W4A4 on Qwen3-8B and Qwen3-32B, respectively. Relative to the evaluated W4A8 baselines, ResServe reduces request latency by 22.4–53.3% with NVFP4 and 35.1–49.3% with MXFP4.

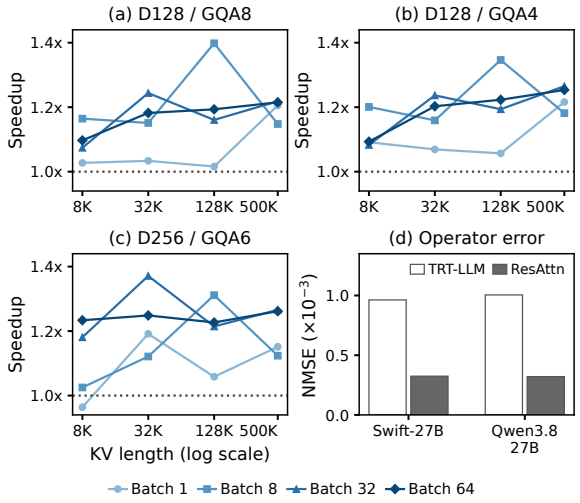

Figure 10. ResAttn versus native BF16-output TRT-LLM with page32 NVFP4 KV: (a–c) B200 speedup; (d) real-input pooled NMSE against the same dequantized KV reference.

Table 1. HTTP latency (ms) with NVFP4 KV, averaged across three independent server processes. ResServe prefill ratios are 0 (NVFP4) and 0.25 (MXFP4). Appendix C.5 and Appendix Table 9 give the protocol and NVFP4 ratio sweep.

Format	Method	TTFT	TPOT	E2E latency
Qwen3-8B				
NVFP4	W4A4	90.90	2.940	840.48
	W4A8	142.87	3.830	1119.39
	ResServe	91.58	3.048	868.76
MXFP4	W4A4	88.59	2.747	789.04
	W4A8	117.05	4.646	1301.81
	ResServe	96.83	2.932	844.49
Qwen3-32B				
NVFP4	W4A4	274.27	8.520	2446.96
	W4A8	686.33	17.863	5241.45
	ResServe	273.96	8.521	2446.91
MXFP4	W4A4	260.06	7.948	2286.71
	W4A8	450.20	17.599	4938.03
	ResServe	309.27	8.601	2502.50

6.4 Task Ablation and Quality

We first isolate M-Ext on the Hard350 subset of LiveCodeBench (LCB) v6 (Jain et al., 2025), using BF16 KV and the same attention implementation. Within each FP4 format, plain and M-Ext share quantized weights, prompts, and generation settings. Selective prefill is disabled in this comparison. Averaged across five models, relative score gains over same-format plain inference are 7.70% for NVFP4 and 18.18% for MXFP4. Total output tokens, including reasoning and final answers across all problems, decrease in all ten model–format pairs, with mean reductions of 4.54% for NVFP4 and 11.81% for MXFP4.

With NVFP4 KV, adding M-Ext to ResAttn improves scores in all ten model–format pairs, with mean relative gains of 8.12% for NVFP4 and 30.97% for MXFP4 across five models. Appendix Table 4 reports component ablations and output-token savings.

System evaluations use a fixed prefill compensation ratio of 0 for NVFP4 and 0.25 for MXFP4 across all five models. The MXFP4 setting is motivated by the retrieval gains from selective linear compensation in Figure 11, discussed later in this section. Table 2 reports LCB, HMMT November 2025, and AIME 2026 results under these configurations. With NVFP4 KV in both configurations, ResServe enables M-Ext and ResAttn, while same-format plain inference uses TensorRT-LLM attention. For each model and task, compared methods use the same prompts, thinking mode, sampling parameters, and output budget. LCB reports pass@1 from one generation per problem. Mathematics evaluations use MathArena (Dekoninck et al., 2026) and report avg@16, the mean correctness over 16 samples per problem.

Across the five models, ResServe improves LCB accuracy over same-format plain inference with mean relative gains of 15.04% for NVFP4 and 53.57% for MXFP4. On LCB, total output tokens decrease by an average of 3.18% for NVFP4 and 9.16% for MXFP4 relative to the same baselines. Mean relative mathematics gains range from 6.02% to 9.15% for NVFP4 and 33.34% to 50.23% for MXFP4. MXFP4 improves mathematics scores in all ten model–task pairs. To assess sensitivity to which questions are included, we perform 10,000 paired bootstrap resamples of questions. For both FP4 formats on all three tasks, the 95% confidence intervals for mean score differences in percentage points across the five models lie above zero. This supports ResServe's average quality gains on code generation and mathematics.

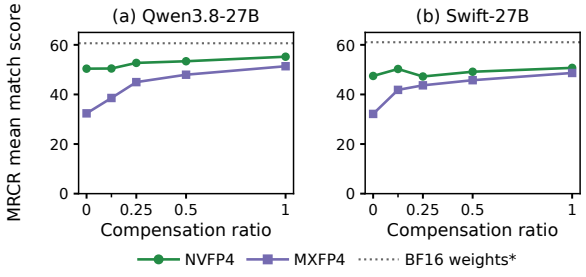

Figure 11. Mean MRCR matching scores (0–100) on Qwen3.8-27B and Swift-27B. All configurations use NVFP4 KV and TRT-LLM attention. Dotted lines (*) are BF16-weight references with the same KV.

Table 2. System accuracy (%): LCB pass@1; HMMT/AIME avg@16. BF16 references use BF16 KV; FP4 rows use NVFP4 KV. ResServe uses prefill ratios 0 (NVFP4) and 0.25 (MXFP4). Bold marks the better same-format score.

Configuration		LCB	HMMT	AIME
Qwen3-4B	BF16	36.57	51.25	63.13
NVFP4	W4A4	23.71	31.88	50.00
	ResServe	**28.86**	**37.92**	**58.75**
MXFP4	W4A4	10.57	14.58	28.13
	ResServe	**20.86**	**33.96**	**50.83**
Qwen3-8B	BF16	45.14	55.83	68.33
NVFP4	W4A4	24.29	43.13	56.25
	ResServe	**35.14**	**48.96**	**61.04**
MXFP4	W4A4	12.00	22.08	36.88
	ResServe	**27.14**	**40.63**	**55.21**
Qwen3-32B	BF16	51.14	64.38	75.63
NVFP4	W4A4	42.29	55.21	**71.46**
	ResServe	**46.29**	**61.67**	71.25
MXFP4	W4A4	31.43	45.83	57.71
	ResServe	**38.00**	**52.50**	**67.08**
Qwen3.8-27B	BF16	84.00	79.38	84.17
NVFP4	W4A4	**79.43**	**80.83**	**88.75**
	ResServe	77.71	78.96	88.33
MXFP4	W4A4	70.86	66.25	72.92
	ResServe	**74.86**	**72.29**	**78.75**
Swift-27B	BF16	83.43	76.25	82.08
NVFP4	W4A4	76.57	75.42	81.46
	ResServe	**77.71**	**78.33**	**85.42**
MXFP4	W4A4	65.71	62.50	67.71
	ResServe	**77.43**	**69.17**	**75.83**

ResServe achieves higher scores than the evaluated W4A8 baselines on Qwen3-4B and Qwen3-8B across all three tasks and both FP4 formats. Scores are broadly comparable on the 27B and 32B models (Appendix Table 7).

Selective linear compensation also improves retrieval on a fixed 300-item MRCR (OpenAI, 2025) 8-needle subset spanning 4–32K contexts. With NVFP4 KV and TRT-LLM attention held fixed, ratio 0.5 improves scores over same-format plain inference across all five models and both FP4 formats. Figure 11 shows the two 27B models; Appendix Figure 17 provides all five.

7 Discussion

7.1 The Execution Cost of Precision Recovery

M-Ext recovers precision at near-plain FP4 latency for the evaluated linear operators with few rows. Both NVFP4 and MXFP4 M-Ext achieve lower latency than the evaluated W4A8 implementations. NVFP4 M-Ext also reduces output error, and MXFP4 M-Ext approaches W4A8 output precision. These results establish efficiently organized FP4 compensation as an alternative to increasing activation bit width for precision recovery. With identical quantized values, M-Ext executes full compensation faster than K-Ext, showing that computation organization changes the cost of recovering the same residue information. For larger prefill operations, selective compensation improves precision over plain FP4 and reduces execution cost relative to full compensation.

Low compensation cost carries through to full-model inference (Table 1). In the evaluated HTTP workload, the quality evaluation's configurations for both FP4 formats add less than 10% request-latency overhead over same-format W4A4 and reduce latency relative to the evaluated W4A8 baselines. Together with the independently measured task-quality gains, these results demonstrate the practical value of FP4 residual compensation.

7.2 Task Quality and Output Length

Precision recovery can improve task quality and reduce output length. Prior work finds that some aggressive quantization settings reduce reasoning accuracy and increase generation length (Lotfi et al., 2026). In the BF16-KV LiveCodeBench ablation, linear compensation improves mean task scores while reducing total output tokens across all model–format pairs. The full system also improves mean LCB accuracy while reducing output tokens on average in both FP4 formats. These results make generation length relevant alongside per-token execution cost when evaluating quantized reasoning.

The benefits of compensation depend on the inference stage and task. Selective compensation improves long-context retrieval, while M-Ext provides low-cost full-residue compensation during reasoning generation. The gains from additional prefill compensation in code generation vary across models, motivating task-aware selection of the compensation configuration.

8 Related Work

Quantization methods reduce weight and activation error (Frantar et al., 2023; Lin et al., 2024; Xiao et al., 2023; Zhao et al., 2024). Higher-precision activation paths use wider floating-point formats (Micikevicius et al., 2022; Mishra et al., 2025), W4A8 execution (Lin et al., 2025), or selective precision allocation (Saxena et al., 2025; Liu et al., 2026). Other work varies precision between prefill and decode (Lu et al., 2026) in disaggregated serving (Zhong et al., 2024). Quantized residues have been used to correct weights (Wang et al., 2025) and activations (Meng et al., 2026). ResServe couples compensation with operator-specific execution, using on-chip result combination and block-local selection to control its runtime cost. Low-precision attention reduces computation

cost (Zhang et al., 2025a;b), while KV-cache compression reduces storage (Liu et al., 2024; Hooper et al., 2024). ResAttn applies M-Ext's compensation execution principle to attention, combining compensation scores before softmax while reusing the existing paged NVFP4 cache.

FlashInfer and FlashDecoding++ adapt execution to input shapes (Ye et al., 2025; Hong et al., 2024), while FlashDecoding exploits parallelism across the KV sequence for attention (Dao et al., 2023). ResServe selects kernel configurations for each input shape, accounting for compensation layouts, Fold costs, and attention reduction costs.

9 Conclusion

ResServe improves block-scaled FP4 task quality with low compensation cost. The system adapts residue computation to the matrix dimensions, combines results on-chip, and selectively compensates prefill channels. It integrates NVFP4/MXFP4 linear computation with paged NVFP4 KV caching. Full-model evaluations demonstrate higher mean coding and mathematics accuracy, alongside retrieval gains from selective linear compensation. On code generation, these quality gains are accompanied by average reductions in output tokens. Low-load NVFP4 serving measurements establish low integration overhead, retaining near-baseline request latency with selective prefill disabled. Efficiently organizing additional low-precision computation offers a practical way to retain low-bit execution efficiency while improving LLM performance on downstream tasks. We plan to extend this principle to grouped GEMM in mixture-of-experts (MoE) models, exploring low-cost compensation under different expert loads.

References

Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebron, F., and Sanghai, S. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In Bouamor, H., Pino, J., and Bali, K. (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 4895–4901, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.298. URL https://aclanthology.org/2023.emnlp-main.298/.

Ashkboos, S., Mohtashami, A., Croci, M. L., Li, B., Cameron, P., Jaggi, M., Alistarh, D., Hoefler, T., and Hensman, J. QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs. In Globerson, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J., and Zhang, C. (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 100213–100240. Curran Associates, Inc., 2024. doi: 10.52202/079017-3180. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/b5b939436789f76f08b9d0da5e81af7c-Paper-Conference.pdf.

Baek, D., Choi, J., Son, J., Bin, K., Choi, S., Moon, K., Jang, M., and Lee, H. FireQ: Fast INT4-FP8 Kernel and RoPE-aware Quantization for LLM Inference Acceleration, July 2025. URL http://arxiv.org/abs/2505.20839.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F. P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W. H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. Evaluating Large Language Models Trained on Code, July 2021. URL http://arxiv.org/abs/2107.03374.

Cook, J., Guo, J., Xiao, G., Lin, Y., and Han, S. Four Over Six: More Accurate NVFP4 Quantization with Adaptive Block Scaling, December 2025. URL http://arxiv.org/abs/2512.02010.

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., and Oh, A. (eds.), *Advances in Neural Information Processing Systems*, volume 35, pp. 16344–16359. Curran Associates, Inc., 2022. doi: 10.52202/068431-1189. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/67d57c32e20fd0a7a302cb81d36e40d5-Paper-Conference.pdf.

Dao, T., Haziza, D., Massa, F., and Sizov, G. Flash-Decoding for long-context inference. PyTorch Blog, October 2023. URL https://pytorch.org/blog/flash-decoding/.

Dekoninck, J., Jovanović, N., Gehrunger, T., Rögnvaldsson, K., Petrov, I., Sun, C., and Vechev, M. Beyond benchmarks: MathArena as an evaluation platform for mathematics with LLMs. 2026. URL https://arxiv.org/abs/2605.00674.

Dettmers, T., Svirschevski, R., Egiazarian, V., Kuznedelev, D., Frantar, E., Ashkboos, S., Borzunov, A., Hoefler, T., and Alistarh, D. SpQR: A Sparse-

Quantized Representation for Near-Lossless LLM Weight Compression. In Kim, B., Yue, Y., Chaudhuri, S., Fragkiadaki, K., Khan, M., and Sun, Y. (eds.), *International Conference on Learning Representations*, volume 2024, pp. 5733–5761, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/file/1787533e171dcc8549cc2eb5a4840eec-Paper-Conference.pdf.

Egiazarian, V., Castro, R. L., Kuznedelev, D., Panferov, A., Kurtic, E., Pandit, S., Marques, A., Kurtz, M., Ashkboos, S., Hoefler, T., and Alistarh, D. Bridging the Gap Between Promise and Performance for Microscaling FP4 Quantization. In *International Conference on Learning Representations*, volume 2026, pp. 113529–113563, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/file/b87bb4f6346d727b265088235e5bc389-Paper-Conference.pdf.

Frantar, E., Ashkboos, S., Hoefler, T., and Alistarh, D. OPTQ: Accurate Quantization for Generative Pre-trained Transformers. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=tcbBPnfwxS.

Guo, D., Yang, D., Zhang, H., Song, J., Wang, P., Zhu, Q., Xu, R., Zhang, R., Ma, S., Bi, X., Zhang, X., Yu, X., Wu, Y., Wu, Z. F., Gou, Z., Shao, Z., Li, Z., Gao, Z., Liu, A., Xue, B., Wang, B., Wu, B., Feng, B., Lu, C., Zhao, C., Deng, C., Ruan, C., Dai, D., Chen, D., Ji, D., Li, E., Lin, F., Dai, F., Luo, F., Hao, G., Chen, G., Li, G., Zhang, H., Xu, H., Ding, H., Gao, H., Qu, H., Li, H., Guo, J., Li, J., Chen, J., Yuan, J., Tu, J., Qiu, J., Li, J., Cai, J. L., Ni, J., Liang, J., Chen, J., Dong, K., Hu, K., You, K., Gao, K., Guan, K., Huang, K., Yu, K., Wang, L., Zhang, L., Zhao, L., Wang, L., Zhang, L., Xu, L., Xia, L., Zhang, M., Zhang, M., Tang, M., Zhou, M., Li, M., Wang, M., Li, M., Tian, N., Huang, P., Zhang, P., Wang, Q., Chen, Q., Du, Q., Ge, R., Zhang, R., Pan, R., Wang, R., Chen, R. J., Jin, R. L., Chen, R., Lu, S., Zhou, S., Chen, S., Ye, S., Wang, S., Yu, S., Zhou, S., Pan, S., Li, S. S., Zhou, S., Wu, S., Yun, T., Pei, T., Sun, T., Wang, T., Zeng, W., Liu, W., Liang, W., Gao, W., Yu, W., Zhang, W., Xiao, W. L., An, W., Liu, X., Wang, X., Chen, X., Nie, X., Cheng, X., Liu, X., Xie, X., Liu, X., Yang, X., Li, X., Su, X., Lin, X., Li, X. Q., Jin, X., Shen, X., Chen, X., Sun, X., Wang, X., Song, X., Zhou, X., Wang, X., Shan, X., Li, Y. K., Wang, Y. Q., Wei, Y. X., Zhang, Y., Xu, Y., Li, Y., Zhao, Y., Sun, Y., Wang, Y., Yu, Y., Zhang, Y., Shi, Y., Xiong, Y., He, Y., Piao, Y., Wang, Y., Tan, Y., Ma, Y., Liu, Y., Guo, Y., Ou, Y., Wang, Y., Gong, Y., Zou, Y., He, Y., Xiong, Y., Luo, Y., You, Y., Liu, Y., Zhou, Y., Zhu, Y. X., Huang, Y., Li, Y., Zheng, Y., Zhu, Y., Ma, Y., Tang, Y., Zha, Y., Yan, Y., Ren, Z. Z., Ren, Z., Sha, Z., Fu, Z., Xu, Z., Xie, Z., Zhang, Z., Hao, Z., Ma, Z., Yan, Z., Wu, Z., Gu, Z., Zhu, Z., Liu, Z., Li, Z., Xie, Z., Song, Z., Pan, Z., Huang, Z., Xu, Z., Zhang, Z., and Zhang, Z. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645(8081):633–638, September 2025. ISSN 1476-4687. doi: 10.1038/s41586-025-09422-z. URL https://www.nature.com/articles/s41586-025-09422-z.

Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. Measuring Mathematical Problem Solving With the MATH Dataset. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, volume 1, 2021. URL https://datasets-benchmarks-proceedings.neurips.cc/paper_files/paper/2021/file/be83ab3ecd0db773eb2dc1b0a17836a1-Paper-round2.pdf.

Hong, K., Dai, G., Xu, J., Mao, Q., Li, X., Liu, J., Chen, K., Dong, Y., and Wang, Y. FlashDecoding++: Faster large language model inference with asynchronization, flat GEMM optimization, and heuristics. In Gibbons, P., Pekhimenko, G., and Sa, C. D. (eds.), *Proceedings of Machine Learning and Systems*, volume 6, pp. 148–161, 2024. URL https://proceedings.mlsys.org/paper_files/paper/2024/file/5321b1dabcd2be188d796c21b733e8c7-Paper-Conference.pdf.

Hooper, C., Kim, S., Mohammadzadeh, H., Mahoney, M. W., Shao, Y. S., Keutzer, K., and Gholami, A. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. In Globerson, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J., and Zhang, C. (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 1270–1303. Curran Associates, Inc., 2024. doi: 10.52202/079017-0040. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/028fcbcf85435d39a40c4d61b42c99a4-Paper-Conference.pdf.

Jain, N., Han, K., Gu, A., Li, W.-D., Yan, F., Zhang, T., Wang, S., Solar-Lezama, A., Sen, K., and Stoica, I. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. In *International Conference on Learning Representations*, volume 2025, pp. 58791–58831, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/file/94074dd5a072d28ff75a76dabed43767-Paper-Conference.pdf.

Jarmusch, A., Graddon, N., and Chandrasekaran, S. Dissecting the NVIDIA Blackwell Architecture with Microbenchmarks, July 2025. URL http://arxiv.

org/abs/2507.10789.

Kim, S., Hooper, C. R. C., Gholami, A., Dong, Z., Li, X., Shen, S., Mahoney, M. W., and Keutzer, K. SqueezeLLM: Dense-and-Sparse Quantization. In Salakhutdinov, R., Kolter, Z., Heller, K., Weller, A., Oliver, N., Scarlett, J., and Berkenkamp, F. (eds.), *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 23901–23923. PMLR, July 2024. URL https://proceedings.mlr.press/v235/kim24f.html.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pp. 611–626. ACM, 2023.

Li, Y., Gong, R., Tan, X., Yang, Y., Hu, P., Zhang, Q., Yu, F., Wang, W., and Gu, S. BRECQ: Pushing the Limit of Post-Training Quantization by Block Reconstruction. In *International Conference on Learning Representations*, 2021. URL https://openreview.net/forum?id=POWv6hDd9XH.

Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., and Han, S. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. *Proceedings of machine learning and systems*, 6:87–100, 2024.

Lin, Y., Tang, H., Yang, S., Zhang, Z., Xiao, G., Gan, C., and Han, S. QServe: W4A8KV4 Quantization and System Co-design for Efficient LLM Serving. In *Proceedings of Machine Learning and Systems*, volume 7. MLSys, 2025. URL https://proceedings.mlsys.org/paper_files/paper/2025/file/fbe2b2f74a2ece8070d8fb073717bda6-Paper-Conference.pdf.

Liu, R., Sun, Y., Zhang, M., Bai, H., Yu, X., Yu, T., Yuan, C., and Hou, L. Quantization Hurts Reasoning? An Empirical Study on Quantized Reasoning Models. In *The Third Conference on Language Modeling*, 2025a. URL https://openreview.net/forum?id=BM192Ps5Nv.

Liu, W., Meng, H., Luo, Y., Zhang, P., and Ma, X. MicroMix: Efficient mixed-precision quantization with microscaling formats for large language models. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=P5OKoZdwlB.

Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., and Hu, X. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In Salakhutdinov, R., Kolter, Z., Heller, K., Weller, A., Oliver, N., Scarlett, J., and Berkenkamp, F. (eds.), *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 32332–32344. PMLR, July 2024. URL https://proceedings.mlr.press/v235/liu24bz.html.

Liu, Z., Zhao, C., Fedorov, I., Soran, B., Choudhary, D., Krishnamoorthi, R., Chandra, V., Tian, Y., and Blankevoort, T. SpinQuant: LLM Quantization with Learned Rotations. In Yue, Y., Garg, A., Peng, N., Sha, F., and Yu, R. (eds.), *International Conference on Learning Representations*, volume 2025, pp. 92009–92032, 2025b. URL https://proceedings.iclr.cc/paper_files/paper/2025/file/e5b1c0d4866f72393c522c8a00eed4eb-Paper-Conference.pdf.

Lotfi, S., Kirichenko, P., Li, S., and Liu, Z. Quantized Reasoning Models Think They Need to Think Longer, but They Do Not, 2026. URL https://arxiv.org/abs/2606.00206v1.

Lu, H., Chen, Z., Fang, G., Ma, X., and Wang, X. MixQuant: Quantized Prefilling, Precise Decoding for Agentic LLMs, May 2026. URL http://arxiv.org/abs/2605.20315.

Meng, H., Luo, Y., Zhao, Y., Liu, W., Zhang, P., and Ma, X. ARCQuant: Boosting NVFP4 Quantization with Augmented Residual Channels for LLMs. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 8609–8623, San Diego, California, United States, 2026. Association for Computational Linguistics. ISBN 979-8-89176-390-6. doi: 10.18653/v1/2026.acl-long.388. URL https://aclanthology.org/2026.acl-long.388/.

Micikevicius, P., Stosic, D., Burgess, N., Cornea, M., Dubey, P., Grisenthwaite, R., Ha, S., Heinecke, A., Judd, P., Kamalu, J., Mellempudi, N., Oberman, S., Shoeybi, M., Siu, M., and Wu, H. FP8 Formats for Deep Learning, September 2022. URL http://arxiv.org/abs/2209.05433.

Mishra, A., Stosic, D., Layton, S., and Micikevicius, P. Recipes for Pre-training LLMs with MXFP8, August 2025. URL http://arxiv.org/abs/2506.08027.

NVIDIA. NVIDIA TensorRT-LLM, n.d.a. URL https://github.com/NVIDIA/TensorRT-LLM.

NVIDIA. PTX ISA 9.4 Documentation, n.d.b. URL https://docs.nvidia.com/cuda/parallel-thread-execution/.

NVIDIA, Abecassis, F., Agrusa, A., Ahn, D., Alben, J., Alborghetti, S., Andersch, M., Arayandi, S., Bjorlin, A., Blakeman, A., Briones, E., Buck, I., Catanzaro, B., Choi, J., Chrzanowski, M., Chung, E., Cui, V., Dai, S., Rouhani, B. D., del Mundo, C., Donia,

D., Eryilmaz, B., Estela, H., Goel, A., Goncharov, O., Guvvala, Y., Hesse, R., Hewett, R., Hum, H., Kapasi, U., Khailany, B., Khona, M., Knight, N., Kondratenko, A., Krashinsky, R., Lanir, B., Layton, S., Lightstone, M., Lo, D., Micikevicius, P., Mishra, A., Moon, T., Narayanan, D., Ni, C., Paithankar, A., Pasumarthi, S., Patel, A., Patwary, M., Poojary, A., Prasad, G., Priyadarshi, S., Qin, Y., Ren, X., Rybakov, O., Sakr, C., Satheesh, S., Sergienko, S., Shamis, P., Shankar, K., Sharma, N., Shoeybi, M., Siu, M., Smelyanskiy, M., Stosic, D., Stosic, D., Su, B.-Y., Sun, F., Tajbakhsh, N., Thomas, S., Tredak, P., Tsykunov, E., Vaithilingam, G., Vavre, A., Venkatesan, R., Waleffe, R., Wan, Q., Wang, H., Wang, M., Wei, L., Wu, H., Wu, E., Wyss, K., Xu, N., Xue, J., Yang, C., Zhai, Y., Zhang, R., Zhu, J., and Zhu, Z. Pretraining Large Language Models with NVFP4, September 2025. URL http://arxiv.org/abs/2509.25149.

NVIDIA Corporation. Blackwell SM100 GEMMs, 2026a. URL https://docs.nvidia.com/cutlass/latest/media/docs/cpp/blackwell_functionality.html.

NVIDIA Corporation. NVIDIA Model Optimizer (ModelOpt), 2026b. URL https://github.com/NVIDIA/Model-Optimizer.

OpenAI. OpenAI MRCR Dataset, April 2025. URL https://huggingface.co/datasets/openai/mrcr.

Qwen Team. Qwen3.8-Max: A new bar for coding and cowork, August 2026. URL https://qwen.ai/blog?id=qwen3.8.

Rouhani, B. D., Garegrat, N., Savell, T., More, A., Han, K.-N., Zhao, R., Hall, M., Klar, J., Chung, E., Yu, Y., Schulte, M., Wittig, R., Bratt, I., Stephens, N., Milanovic, J., Brothers, J., Dubey, P., Cornea, M., Heinecke, A., Rodriguez, A., Langhammer, M., Deng, S., Naumov, M., Micikevicius, P., Siu, M., and Verrilli, C. OCP Microscaling Formats (MX) Specification. Technical Report Version 1.0, Open Compute Project, September 2023. URL https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf.

Saxena, U., Sharify, S., Roy, K., and Wang, X. ResQ: Mixed-precision quantization of large language models with low-rank residuals. In Singh, A., Fazel, M., Hsu, D., Lacoste-Julien, S., Berkenkamp, F., Maharaj, T., Wagstaff, K., and Zhu, J. (eds.), *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 53095–53114. PMLR, July 2025. URL https://proceedings.mlr.press/v267/saxena25b.html.

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., and Scialom, T. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*, volume 36, pp. 68539–68551. Curran Associates, Inc., 2023. doi: 10.52202/075280-2997. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/d842425e4bf79ba039352da0f658a906-Paper-Conference.pdf.

Shao, W., Chen, M., Zhang, Z., Xu, P., Zhao, L., Li, Z., Zhang, K., Gao, P., Qiao, Y., and Luo, P. OmniQuant: Omnidirectionally Calibrated Quantization for Large Language Models. In Kim, B., Yue, Y., Chaudhuri, S., Fragkiadaki, K., Khan, M., and Sun, Y. (eds.), *International Conference on Learning Representations*, volume 2024, pp. 45472–45496, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/file/c6483c8a68083af3383f91ee0dc6db95-Paper-Conference.pdf.

Sun, Y., Liu, R., Bai, H., Bao, H., Zhao, K., Li, Y., Hu, J., Yu, X., Hou, L., Yuan, C., Jiang, X., Liu, W., and Yao, J. FlatQuant: Flatness Matters for LLM Quantization. In Singh, A., Fazel, M., Hsu, D., Lacoste-Julien, S., Berkenkamp, F., Maharaj, T., Wagstaff, K., and Zhu, J. (eds.), *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 57587–57613. PMLR, July 2025. URL https://proceedings.mlr.press/v267/sun25l.html.

UkisAI. Swift 1.5 qwen3.8-27B, 2026. URL https://huggingface.co/ukisai/Swift-1.5-Qwen3.8-27b.

Wang, F., Tan, X., Huang, B., Zhang, Y., Wang, G., Cong, P., and Yang, T. Fairy2i: Training Complex LLMs from Real LLMs with All Parameters in $\{\pm 1, \pm i\}$, December 2025. URL http://arxiv.org/abs/2512.02901.

Wang, Y., Ma, X., Zhang, G., Ni, Y., Chandra, A., Guo, S., Ren, W., Arulraj, A., He, X., Jiang, Z., Li, T., Ku, M., Wang, K., Zhuang, A., Fan, R., Yue, X., and Chen, W. MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark. In *Advances in Neural Information Processing Systems*, volume 37, pp. 95266–95290. Curran Associates, Inc., 2024. doi: 10.52202/079017-3018. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/ad236edc564f3e3156e1b2feafb99a24-Paper-Datasets_and_Benchmarks_Track.pdf.

Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., and Han, S. SmoothQuant: Accurate and efficient post-training quantization for large language models. In Krause, A., Brunskill, E., Cho, K., Engelhardt, B.,

Sabato, S., and Scarlett, J. (eds.), *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 38087–38099. PMLR, July 2023. URL https://proceedings.mlr.press/v202/xiao23c.html.

Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Zhou, J., Lin, J., Dang, K., Bao, K., Yang, K., Yu, L., Deng, L., Li, M., Xue, M., Li, M., Zhang, P., Wang, P., Zhu, Q., Men, R., Gao, R., Liu, S., Luo, S., Li, T., Tang, T., Yin, W., Ren, X., Wang, X., Zhang, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Zhang, Y., Wan, Y., Liu, Y., Wang, Z., Cui, Z., Zhang, Z., Zhou, Z., and Qiu, Z. Qwen3 Technical Report, May 2025. URL http://arxiv.org/abs/2505.09388.

Yao, Z., Dong, Z., Zheng, Z., Gholami, A., Yu, J., Tan, E., Wang, L., Huang, Q., Wang, Y., Mahoney, M. W., and Keutzer, K. HAWQ-V3: Dyadic Neural Network Quantization. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pp. 11875–11886. PMLR, 2021. URL https://proceedings.mlr.press/v139/yao21a.html.

Ye, Z., Chen, L., Lai, R., Lin, W., Zhang, Y., Wang, S., Chen, T., Kasikci, B., Grover, V., Krishnamurthy, A., and Ceze, L. FlashInfer: Efficient and customizable attention engine for LLM inference serving. In Zaharia, M., Joshi, G., and Lin, Y. C. (eds.), *Proceedings of the Eighth Conference on Machine Learning and Systems, MLSys 2025, Santa Clara, CA, USA, May 12-15, 2025*. mlsys.org, 2025. URL https://openreview.net/forum?id=RXPofAsL8F.

Zhang, C., Cheng, J., Constantinides, G. A., and Zhao, Y. LQER: Low-Rank Quantization Error Reconstruction for LLMs. In Salakhutdinov, R., Kolter, Z., Heller, K., Weller, A., Oliver, N., Scarlett, J., and Berkenkamp, F. (eds.), *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 58763–58779. PMLR, July 2024. URL https://proceedings.mlr.press/v235/zhang24j.html.

Zhang, J., Huang, H., Zhang, P., Wei, J., Zhu, J., and Chen, J. SageAttention2: Efficient attention with thorough outlier smoothing and per-thread INT4 quantization. In Singh, A., Fazel, M., Hsu, D., Lacoste-Julien, S., Berkenkamp, F., Maharaj, T., Wagstaff, K., and Zhu, J. (eds.), *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 75097–75119. PMLR, July 2025a. URL https://proceedings.mlr.press/v267/zhang25ae.html.

Zhang, J., Wei, J., Wang, H., Zhang, P., Xu, X., Huang, H., Jiang, K., Chen, J., and Zhu, J. SageAttention3: Microscaling FP4 attention for inference and an exploration of 8-bit training. In Belgrave, D., Zhang, C., Lin, H., Pascanu, R., Koniusz, P., Ghassemi, M., and Chen, N. (eds.), *Advances in Neural Information Processing Systems*, volume 38, Main Conference, pp. 53901–53931. Curran Associates, Inc., 2025b. doi: 10.52202/085713-1799. URL https://proceedings.neurips.cc/paper_files/paper/2025/file/4db397e0f760cc573c681e81a01a3dba-Paper-Conference.pdf.

Zhao, Y., Lin, C.-Y., Zhu, K., Ye, Z., Chen, L., Zheng, S., Ceze, L., Krishnamurthy, A., Chen, T., and Kasikci, B. Atom: Low-bit quantization for efficient and accurate LLM serving. *Proceedings of Machine Learning and Systems*, 6:196–209, 2024.

Zheng, Z., Song, X., and Liu, C. MixLLM: LLM Quantization with Global Mixed-precision between Output-features and Highly-efficient System Design. In Chowdhery, A. and Jia, Z. (eds.), *Proceedings of Machine Learning and Systems*, volume 8, pp. 1771–1785. MLSys, 2026. URL https://proceedings.mlsys.org/paper_files/paper/2026/file/a66caa1703fe34705a4368c3014c1966-Paper-Conference.pdf.

Zhong, Y., Liu, S., Chen, J., Hu, J., Zhu, Y., Liu, X., Jin, X., and Zhang, H. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In Gavrilovska, A. and Terry, D. B. (eds.), *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024*, pp. 193–210. USENIX Association, 2024. URL https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin.

APPENDICES

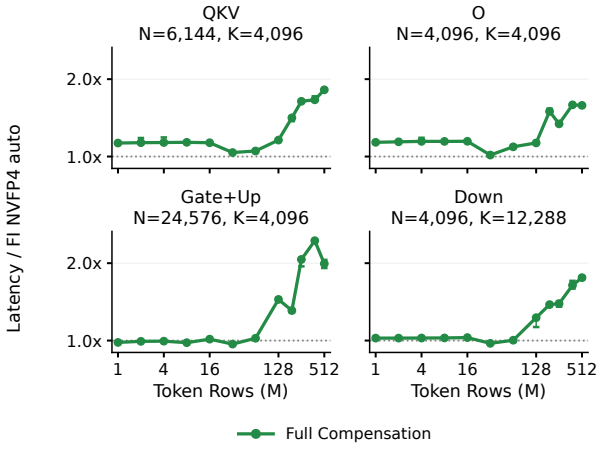

Figure 12. Full-compensation latency relative to plain FlashInfer NVFP4 auto on RTX 5090; lower is better.

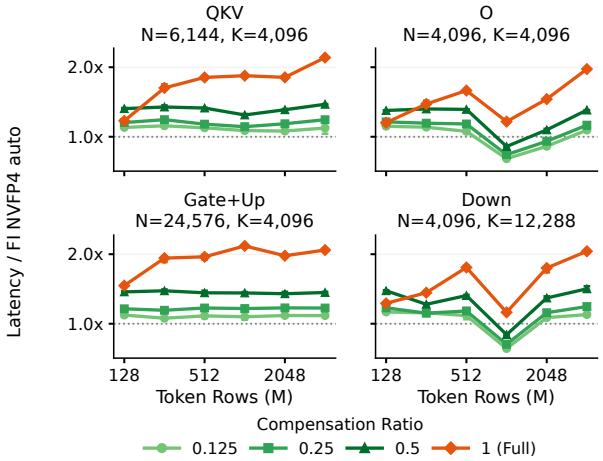

Figure 13. Selective prefill latency relative to plain FlashInfer NVFP4 auto on RTX 5090; lower is better. Full compensation is included for reference.

A NVFP4 LINEAR KERNELS ON RTX 5090

We extend the main text's evaluation of NVFP4 linear kernels to RTX 5090 (SM120), measuring M-Ext and selective prefill overhead and comparing with ARCQuant. We measure four Qwen3-8B projections against FlashInfer's auto-selected plain NVFP4 baseline. Timing includes online activation quantization and GEMM/Fold; Section C.1 gives the measurement protocol.

A.1 Linear-Layer Latency

M-Ext remains close to plain NVFP4 latency at small row counts (Figure 12). For $M = 1$–64, its latency is 0.95–$1.20\times$ that of plain FlashInfer, with a cross-configuration median of $1.04\times$.

A.2 Selective Prefill and ARCQuant Comparison

Selective prefill reduces compensation overhead at larger row counts (Figure 13). Median latencies relative to plain FlashInfer are $1.12\times$, $1.19\times$, and $1.40\times$ for compensation ratios 0.125, 0.25, and 0.5, respectively, versus $1.81\times$ for full compensation.

ResServe is faster than ARCQuant at matching compensation ratios (Meng et al., 2026) (Figure 14). M-Ext achieves a median speedup of $4.05\times$ for $M = 1$–64. K-Ext is faster in all 72 selective configurations, with speedups of 1.24–$2.71\times$ (median $1.99\times$).

Both paths include online activation quantization and GEMM. ARCQuant's implementation computes a global activation scale for each quantization call. Our timing excludes the global amax reduction and scale computation but includes activation prescaling and channel reordering. This exclusion favors ARCQuant by omitting its per-call scale preparation overhead. FlashInfer and ResServe use autotuning; ARCQuant uses its fixed compiled kernels.

B IMPLEMENTATION DETAILS

B.1 Block-Local Selection and Residue Packing

Partial compensation selects a fixed number of channels within each 8-channel group, using ratios $\rho \in \{0.125, 0.25, 0.5\}$. Each layer uses one ratio, with every group contributing $k = 8\rho$ channels. Full compensation includes all channels. Calibration scores channels by their mean absolute activation m_i and selects the top-k within each group $G_g = \{8g, \ldots, 8g + 7\}$:

$$\mathcal{S} = \bigcup_g \mathrm{TopK}\big(\{m_i : i \in G_g\},\, k\big). \tag{7}$$

Calibration uses 1,024 samples with a sequence-length cap of 2,048, batch size 8, and seed 42. One 8-bit mask per group records the selected positions and remains fixed at inference.

The selected operands are quantized as $\hat{\mathbf{R}}_{\mathcal{S}} = \mathrm{QDQ}(\mathbf{R}_{\mathcal{S}})$ and $\hat{\mathbf{W}}_{\mathcal{S}} = \mathrm{QDQ}(\mathbf{W}_{\mathcal{S}})$. The same $\mathcal{S}$ pairs the selected residue channels with their corresponding weight rows. Figure 15 illustrates the augmented GEMM of Eq. 6 for NVFP4 prefill with partial compensation.

B.2 NVFP4 Quantize-and-Extend

NVFP4 combines an FP8 block scale with an FP32 tensor scale. Each 16-element block shares an E4M3 scale;

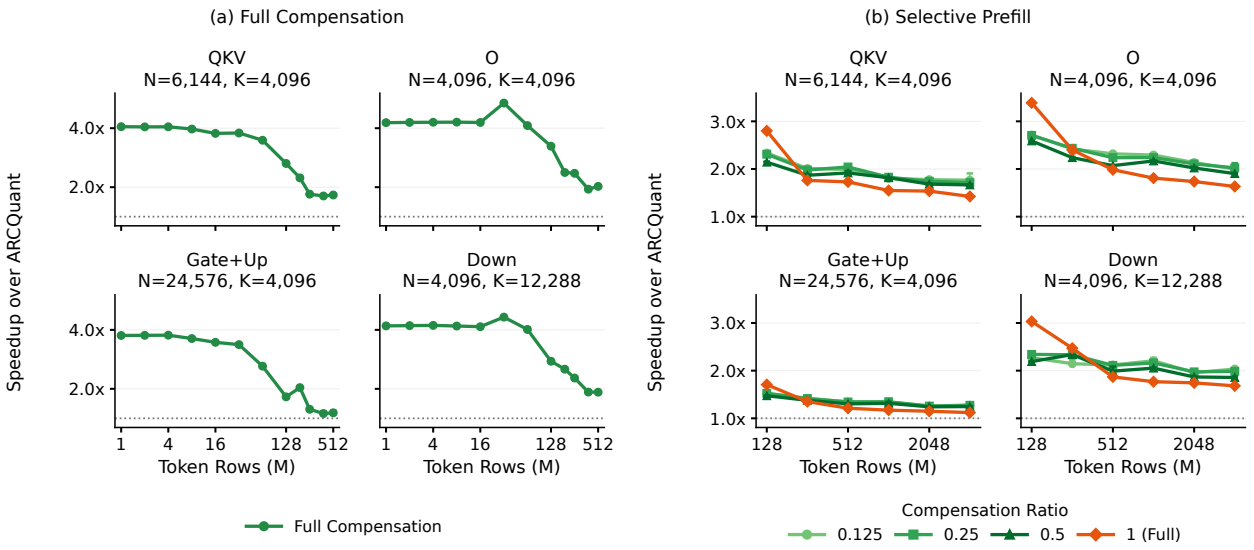

Figure 14. ResServe speedup over ARCQuant on RTX 5090 at matching compensation ratios; higher is better.

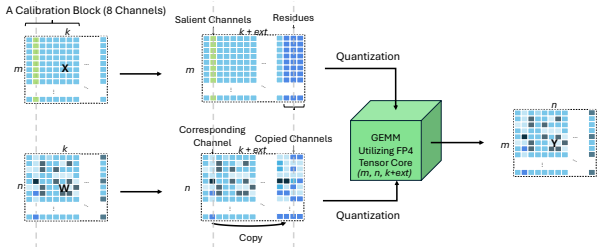

Figure 15. Selective K-Ext pairs runtime residue channels with weight rows prepared offline in an augmented FP4 GEMM.

the tensor scale supplies the common range (NVIDIA et al., 2025; Cook et al., 2025). The maximum magnitudes of E2M1 and E4M3 are 6 and 448, respectively. When the tensor scale is determined from the current input range, each block B_i in tensor $\mathbf{X}$ uses

$$\mathrm{Max}_i = \max_{x \in B_i} |x|, \qquad s_i = \frac{\mathrm{Max}_i}{6}, \qquad \alpha = \frac{\max_i s_i}{448}, \tag{8}$$

where α is the tensor-level scale. The stored FP8 block scale encodes the normalized value s_i/α. Its effective dequantization scale includes α:

$$s_{i,\mathrm{FP8}} = \mathrm{round}_{\mathrm{E4M3}}(s_i/\alpha), \qquad \tilde{s}_i = \alpha s_{i,\mathrm{FP8}}. \tag{9}$$

For $x \in B_i$, FP4 quantization and reconstruction use this effective scale:

$$x_{\mathrm{FP4}} = \mathrm{round}_{\mathrm{E2M1}}(x/\tilde{s}_i), \qquad \hat{x} = \tilde{s}_i x_{\mathrm{FP4}}. \tag{10}$$

The fused kernel generates base and residue operands in one pass. It receives a precomputed tensor scale and computes block scales at runtime. The NVFP4 vLLM integration loads calibrated tensor scales from the checkpoint and reuses them during inference. We describe the pack8 implementation below, which assigns eight input elements to each thread. Two adjacent threads use a warp shuffle to compute the maximum magnitude for their shared 16-element scale block. The kernel quantizes inputs to FP4, then decodes and rescales them to compute residues in registers without writing reconstructed activations to memory.

The selected residues form a separate NVFP4 operand with its own block scales and share the base operand's tensor scale. Each thread contributes k residues, so a scale group spans $16/k$ consecutive threads. Warp shuffles gather scaled residues into eight-value chunks, which are then quantized and packed into 32-bit words for coalesced writes. Algorithm 1 summarizes this partial-compensation mapping; REDUCEMAX reduces over the indicated number of adjacent lanes within one warp.

C EVALUATION PROTOCOLS

C.1 Kernel Timing

Linear layers Linear-layer timing includes online activation quantization and GEMM/Fold to BF16 output. On B200, M-Ext and K-Ext cover 22 and 17 projection shapes, respectively, using shared synthetic BF16 inputs and weights. In selective prefill comparisons, full compensation denotes a compensation ratio of 1.0. Timing uses CUDA Graphs with cold-buffer rotation.

M-Ext and plain FI CuTeDSL use the same FP4 format;

Algorithm 1 Residue NVFP4 quantization: eight inputs per thread.

Input: **bf16** $x[8]$ (8 activations; or fp16), **uint8** $mask$, **fp32** α (tensor scale)
Output: (to global memory) main: 8 **fp4** $+$ **fp8** SF (1 per 2 lanes); residue: 8ρ **fp4** $+$ **fp8** SF (1 per $16/k$ lanes)
// on-lane bf16/fp16 $\rightarrow$ fp32
fp32 $xf[8] = \text{cast}(x)$
// per-16 amax: per-lane max, then reduce over 2 lanes (8 elts each)
fp32 $amax = \textsc{ReduceMax}_2(\max_i |xf[i]|)$
// BLOCKSCALE: fp8 sf (one per 2 lanes) and the fp32 multiplier $scale$
fp8 sf, **fp32** $scale = \textsc{BlockScale}(\alpha, amax)$
if $lane \bmod 2 == 0$: store sf
// QDQ: fp4 codes + fp32 dequant
fp4 $q[8]$, **fp32** $x_q[8] = \text{QDQ}(scale \cdot xf)$; store q
// residue over all 8 channels, in registers
for $i = 0..7$: **fp32** $r[i] = xf[i] - x_q[i]/scale$
int $k = \textsc{popcount}(mask)$; **fp32** $r_sel[k] = \textsc{Select}(r, mask)$
// per-16 residue amax: reduce over $16/k$ lanes (k residues each)
fp32 $r_amax = \textsc{ReduceMax}_{16/k}(\max_i |r_sel[i]|)$
// BLOCKSCALE: fp8 sf_r (one per $16/k$ lanes) and fp32 $scale_r$
fp8 sf_r, **fp32** $scale_r = \textsc{BlockScale}(\alpha, r_amax)$
if $lane \bmod (16/k) == 0$: store sf_r
// scale residues before gathering across lanes
fp32 $r_scaled[k] = scale_r \cdot r_sel$
// gather eight consecutive residues for each writer lane
fp32 $chunk[8] = \textsc{WarpGather8}(r_scaled)$
// quantize eight values to FP4 and pack into one 32-bit word
uint32 $c = \textsc{QuantizePack8}(chunk)$
$w = 32\rho$; **if** $lane < w$: store c

W4A8 baselines use FI/TRT MXFP4/MXFP8 or TRT NVFP4/FP8. The NVFP4 and TRT FP8 linear operator tests precompute global activation scales from each test input's maximum absolute value. Timing excludes this computation but includes online block-scale generation for NVFP4 and MXFP8.

Kernel selection differs across the evaluated backends. M-Ext and plain CuTeDSL use FI autotuning, while K-Ext uses FI automatic GEMM selection. FI MXFP4/MXFP8 uses its default 1-SM tactic. TRT NVFP4/FP8 uses native selection, whereas TRT MXFP4/MXFP8 uses FI tactic tuning. FI CUTLASS NVFP4 is an additional backend reference.

The B200 projection-shape sweeps normalize latency to one common baseline per format. The NVFP4 sweep uses FI NVFP4 CuTeDSL; the MXFP4 sweep uses FI MXFP4 stock. Points show the median paired latency ratio across five independent processes; bars span the minimum and maximum. Cross-shape speedup summaries take the median of paired baseline/M-Ext ratios per shape, then the median across 22 distinct shapes.

The RTX 5090 linear comparisons use cold-L2 CUDA Graph timing across three independent processes. Global-scale preparation and offline weight packing are excluded for all methods. Each point is the median paired latency ratio across processes; bars show their range. Reported summary medians aggregate these per-configuration ratios.

Decode attention Decode-attention timing includes query conversion and attention, but excludes cache construction and KV append. ResAttn and the TRT-LLM baseline with native BF16 output receive identical BF16 queries and prepared page32 NVFP4 KV with global scales of 1.0. Timing uses warm buffers; the baseline uses FlashInfer's native dispatcher and default TRT-LLM selection. KV-head counts are 8, 8, and 4 in panel order.

Each speedup point is the median of three within-process baseline/ResAttn latency ratios. Overall summaries take the median across configurations using all process records.

Full-compensation comparison Figure 16 compares complete NVFP4 paths on SM100 with $N = K = 4096$, using synthetic BF16 inputs and weights. Both paths use pack8 quantization and identical base, residue, and weight codes and scales. K-Ext concatenates along K and duplicates weights offline; M-Ext uses paired rows. Each path independently selects from 128×64 and 128×128 tiles with three pipeline stages and single-CTA clusters. K-Ext also selects whether to swap operands.

Timing includes online quantization, packing, and GEMM/Fold, excluding global-scale preparation and offline weight processing. Cold-buffer CUDA Graph measurements use three independent processes, with per-process medians paired before aggregating speedups. Outputs are checked against a common FP32 reference reconstructed from the represented FP4 values.

C.2 Operator Precision

Linear precision measures error against FP32 matrix multiplication of the original BF16 operands, with TF32 disabled. Within each cohort, all implementations receive identical captured activations and original BF16 weights, prepared in each backend's supported quantized representation. Plain FP4 and same-format M-Ext share quantized weights; W4A8 uses its backend's representation. Output error includes both weight and activation quantization.

The linear studies cover QKV, output, gate/up, and down projections using real decode and prefill inputs. Decode uses 72 inputs from Qwen3-32B; prefill uses 192

input groups per model from Swift-27B and Qwen3.8-27B. The prefill inputs come from AIME, HMMT, and MRCR prompts. The prefill study excludes hybrid linear-attention projections.

Attention precision uses two references, with and without KV quantization loss. The main reference is FP32 attention on the original query and the same dequantized NVFP4 KV, measuring computation error across the complete path. The additional BF16-KV reference also includes cache quantization loss. Both comparisons use 192 matched decode instances per model from Swift-27B and Qwen3.8-27B.

Precision metrics aggregate squared errors before normalization. Pooled MSE divides total squared error by the number of output elements; pooled NMSE divides it by total reference energy. Both attention references use pooled NMSE.

C.3 Reasoning Tasks

The reasoning evaluations compare linear compensation under BF16 KV and system components under NVFP4 KV. The latter uses page size 32 and global K/V scales of 1.0. TRT-LLM uses native BF16 output, and ResAttn replaces decode attention. M-Ext applies at scheduled row counts up to 128, including eligible small prefill chunks.

LCB reports pass@1, while mathematics scores report avg@16: mean correctness over 16 samples per problem. LCB v6 uses the 350 hard problems in release_v6, with official scoring and one generation per problem. HMMT November 2025 and AIME 2026 each contain 30 problems from MathArena (Dekoninck et al., 2026), scored with its official parser. Length-limited outputs remain in the scores. LCB token savings include reasoning and final answers across all 350 problems.

For system evaluations, relative score gains and LCB output-token reductions are computed against each model's same-format plain baseline, then averaged equally across the five models. For the BF16-KV M-Ext ablation, relative score gains and output-token reductions are computed against each model's same-format plain baseline, then averaged equally across the five models. For the NVFP4-KV component ablation, mean relative score gains from adding M-Ext use ResAttn alone as the baseline, with selective prefill disabled in both configurations.

Generation settings are matched across methods within each model. Qwen3-4B/8B/32B use thinking mode at temperature 0.6; Qwen3.8-27B and Swift-27B use medium-effort thinking at temperature 1.0. Both LCB and mathematics runs use top-p 0.95 and top-k 20. LCB uses seed 42 for all five models. The 16 mathematics samples use seeds 42–57 for Qwen3 and 0–15 for the two 27B models, with the same seed set across compared methods. Their context limits are 40,960 tokens for Qwen3 and 262,144 for the two 27B models.

We additionally evaluate W4A8 baselines on code generation and mathematics tasks. Table 7 compares these baselines with plain W4A4 and ResServe across five models, using the task cohorts, scoring definitions, model-specific sampling settings, and seed sets described above.

C.4 Retrieval Task

MRCR (OpenAI, 2025) compares linear compensation while holding the KV format and attention implementation fixed. All arms, including the BF16-weight reference, use page32 NVFP4 KV with global K/V scales of 1.0 and TRT-LLM native BF16-output attention. Ratio 0 denotes uncompensated inference; nonzero ratios apply the checkpoint's compensation configuration in both prefill and decode, without separately enabling decode M-Ext. Ratio 1 retains the checkpoint's full-compensation path. ResAttn is not used.

All configurations use the same 300-item, 8-needle subset spanning 4–32K contexts. We use greedy, non-thinking generation with a 2,048-token output limit and one run per configuration. Scores use the official prefix check and SequenceMatcher, averaged equally over all 300 answers and scaled to 0–100. Length-stopped and empty answers remain included.

C.5 End-to-End Latency

We measure client-side streaming HTTP latency with `vllm bench serve`, using one SM100 GPU per model. Synthetic requests contain exactly 8,192 input and 256 output tokens, with EOS ignored. Arrivals follow a Poisson process at 1 request/s with seed 17 and a client concurrency cap of 32. Each of three independent server processes serves 32 warmup requests followed by 128 measured requests. We report the mean of the three process means.

We evaluate NVFP4 and MXFP4 weights with page32 NVFP4 KV caches in a controlled comparison. The baseline uses upstream vLLM linear auto dispatch with a TRT BF16-output attention adapter. ResServe at ratio 0 uses the same checkpoint with M-Ext and ResAttn; nonzero ratios use selective residue exports. M-Ext follows the production row-count threshold rather than a strict prefill/decode switch. FlashInfer autotuning is enabled for the W4A4 and ResServe configurations.

The runtime uses vLLM 0.26.0, Torch 2.11.0, adapted FlashInfer 0.7.0, and CUTLASS DSL 4.8.0, with full and piecewise CUDA Graphs. Chunked prefill is

Table 3. Linear-operator error on Qwen3-32B (lower is better). Bold values indicate the lowest error within each format.

Implementation	Compute	MSE	NMSE
NVFP4			
FI plain	W4A4	0.295745	0.008401
M-Ext	W4A4	**0.246697**	**0.007007**
TRT NVFP4/FP8	W4A8	0.328123	0.009320
MXFP4			
FI plain	W4A4	0.790377	0.022450
M-Ext	W4A4	0.630280	0.017903
FI MXFP4/MXFP8	W4A8	**0.625482**	**0.017767**
TRT MXFP4/MXFP8	W4A8	**0.625482**	**0.017767**

enabled and prefix caching disabled. We integrate TensorRT-LLM's NVFP4/FP8 GEMM, corresponding to the TRT NVFP4/FP8 baseline in the kernel evaluation, into vLLM for the W4A8 system comparison. The NVFP4 W4A8 measurements use TensorRT-LLM 1.3.0rc24 with static per-tensor FP8 activation scales and GEMM configurations selected by TensorRT-LLM's native heuristic. The MXFP4 W4A8 baseline uses TensorRT-LLM's MXFP4/MXFP8 GEMM with online block-scaled MXFP8 activation quantization and FlashInfer tactic tuning. W4A8 baselines are measured separately using the same framework versions and HTTP workload protocol.

HTTP latency includes queuing, execution, and client/transport overhead. Output throughput is limited by the offered request rate, not maximum serving capacity.

D Additional Evaluation Results

D.1 Kernel Results

Table 3 reports linear-operator error on Qwen3-32B under the protocol in Appendix C.2.

Figures 18–20 provide additional per-shape linear-latency results on B200. Cross-shape summaries use the full measured shape sets described in Appendix C.1.

Across $M = 1$–64, the median MXFP4 M-Ext/plain latency ratio ranges from 1.06 to 1.14. Over the same row counts, median speedups are 2.31–3.22× over FI MXFP4/MXFP8 and 1.55–1.96× over TRT MXFP4/MXFP8.

ResAttn also reduces total attention error relative to TRT-LLM when the reference uses the original BF16 KV, including cache quantization loss. ResAttn reduces pooled NMSE from 0.004595 to 0.003907 on Swift-27B (14.98%). On Qwen3.8-27B, it reduces NMSE from 0.004706 to 0.003941 (16.24%).

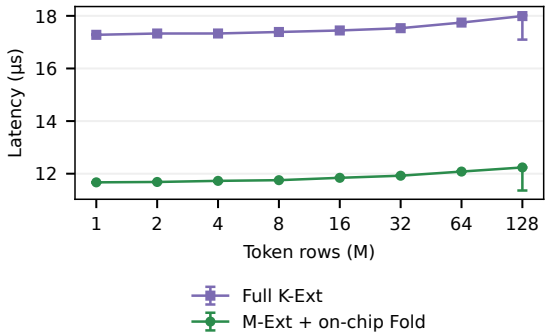

Figure 16. Full NVFP4 compensation on SM100 with $N = K = 4096$: M-Ext versus K-Ext with offline duplicated weights. Timing includes online activation quantization, packing, and GEMM/Fold. Points show medians across three processes; bars show their range.

D.2 LCB Ablation

Table 4 reports LCB pass@1 scores and output-token savings across five models under the protocol in Appendix C.3. The BF16-KV block isolates M-Ext; the NVFP4-KV block compares M-Ext and ResAttn, with an additional MXFP4 row using selective prefill at ratio 0.25.

The MXFP4 prefill comparison holds M-Ext and ResAttn fixed. Increasing the ratio from 0 to 0.25 improves three models' scores and lowers two; all five final system scores exceed the same-format baseline. The uniform ratio follows the retrieval-based choice described in Section 6.4.

Token savings include reasoning and final answers, relative to the same-model, same-format baseline within each KV block. Negative values indicate longer outputs; dashes denote unreported savings.

D.3 System Quality with W4A8 Baselines

Table 7 extends the main text's system comparison with NVFP4/FP8 and MXFP4/MXFP8 W4A8 baselines across all five models. It retains the BF16 references, plain W4A4 results, and fixed ResServe configurations. LCB reports Hard350 pass@1; HMMT November 2025 and AIME 2026 report average correctness over 16 samples per problem. Appendix C.3 specifies the additional baselines and their evaluation settings.

D.4 Retrieval Quality Across Five Models

Figure 17 extends the main text's MRCR comparison to all five models using the protocol in Appendix C.4. Full compensation improves matching scores over same-format plain inference in all ten model–format pairs.

Table 4. LCB Hard350 pass@1 scores and output-token savings (%), one generation per problem. Bold marks the best score within each model, FP4 format, and KV block.

			Qwen3-4B		Qwen3-8B		Qwen3-32B		Qwen3.8-27B		Swift-27B	
Weights	Configuration	Prefill ratio	Score	Tokens saved	Score	Tokens saved	Score	Tokens saved	Score	Tokens saved	Score	Tokens saved
BF16	BF16 KV	—	36.57	—	45.14	—	51.14	—	84.00	—	83.43	—
(a) BF16 KV: M-Ext ablation												
NVFP4	Baseline	0	25.14	0.00	32.00	0.00	41.71	0.00	**77.43**	0.00	81.14	0.00
	Baseline + M-Ext	0	**28.86**	6.04	**36.29**	4.52	**45.43**	5.25	**77.43**	3.23	**82.29**	3.67
MXFP4	Baseline	0	19.71	0.00	17.43	0.00	**35.14**	0.00	73.14	0.00	72.57	0.00
	Baseline + M-Ext	0	**26.57**	12.75	**25.71**	10.43	34.57	4.94	**76.86**	18.61	**76.29**	12.35
(b) NVFP4 KV: component ablation												
BF16	TRT-LLM	—	32.86	—	40.29	—	46.86	—	85.43	—	84.57	—
	ResAttn	—	33.14	—	40.00	—	48.29	—	84.29	—	85.14	—
NVFP4	Baseline (TRT-LLM)	0	23.71	0.00	24.29	0.00	42.29	0.00	**79.43**	0.00	76.57	0.00
	ResAttn	0	26.00	8.13	30.29	1.06	41.71	-3.33	76.86	-0.67	76.57	-5.87
	ResAttn + M-Ext	0	**28.86**	11.31	**35.14**	6.25	**46.29**	-1.25	77.71	0.68	**77.71**	-1.12
MXFP4	Baseline (TRT-LLM)	0	10.57	0.00	12.00	0.00	31.43	0.00	70.86	0.00	65.71	0.00
	ResAttn	0	14.86	3.30	13.14	3.69	33.71	-1.65	74.00	0.49	63.43	-4.15
	ResAttn + M-Ext	0	**23.43**	15.40	22.29	9.36	34.86	5.46	**78.57**	16.52	74.86	13.60
	ResAttn + M-Ext	0.25	20.86	13.98	**27.14**	5.51	**38.00**	5.42	74.86	20.88	**77.43**	0.03

Gains range from 3.42 to 19.00 points for MXFP4 and from 1.23 to 4.82 points for NVFP4.

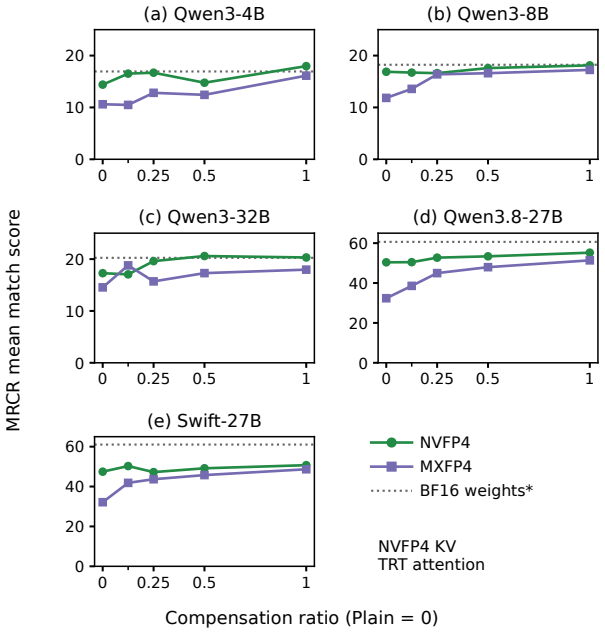

Figure 17. Mean MRCR matching scores (0–100) across five models. All configurations use NVFP4 KV and TRT-LLM attention. Dotted lines (*) are BF16-weight references with the same KV. Plain is at ratio 0; y-axis ranges differ.

D.5 Checkpoint Storage

Selective compensation trades additional checkpoint storage for precision recovery. Table 8 reports complete checkpoint sizes for five models, relative to plain checkpoints of the same model and FP4 format. Sizes count the logical bytes of deployment files, including weight shards, quantization metadata, channel masks, configuration, and tokenizer files. This measures checkpoint storage rather than runtime GPU memory. At ratio 0.125, storage increases by 7.67–10.66% for NVFP4 and 7.51–10.57% for MXFP4. At ratio 0.25, the increases are 15.32–21.30% and 14.99–21.12%, respectively. The main system evaluations use the plain checkpoint at NVFP4 ratio 0 and the expanded checkpoint at MXFP4 ratio 0.25.

Checkpoint growth differs from the channel selection ratio because unquantized tensors and fixed files remain unchanged, while alignment can add padding. For Qwen3-4B MXFP4 at ratio 0.125, an extended width of 2,880 is padded to 2,944.

E Layer-Wise Compensation: Configuration and Evaluation

E.1 Compensation-Ratio Planner

An optional offline planner selects each layer's NVFP4 prefill compensation ratio under an effective-bit budget. Candidate ratios are 0, 0.125, 0.25, 0.5, and 1.0. Calls with few rows use full-residue M-Ext independently of

this plan.

Candidate scores estimate quantization damage using either gradient-weighted error in layer outputs or KL divergence between the output distributions of the reference and candidate models. Gradient-weighted reconstruction also appears in BRECQ (Li et al., 2021). The KL option evaluates one layer or coupled layer group at a time, leaving other searchable layers unquantized. It requires forward passes but no loss gradients.

The planner selects one ratio per layer to minimize the total score within the budget. Effective-bit costs account for base and compensated FP4 computation, weighted by layer size; they do not represent physical storage. As in budgeted mixed-precision allocation (Yao et al., 2021), an offline integer linear program uses precomputed scores and costs, without additional forward passes.

E.2 Reasoning Quality with AWQ

Residue compensation acts on activations and can be combined with offline weight optimization such as AWQ (Lin et al., 2024).

Table 5 reports results combining AWQ with residue compensation for Qwen3-4B and Qwen3-8B at budgets 4.5, 5.0, and 5.5, with layer-wise ratios selected by the planner. Both models run in thinking mode through vLLM with a BF16 KV cache. The benchmarks are the full LiveCodeBench v6 set (LCB, pass@1) (Jain et al., 2025), MATH500 (Hendrycks et al., 2021), and the Math subset of MMLU-Pro (Wang et al., 2024).

Linear compensation improves LCB scores over AWQ alone at all three budgets. The gains are 1.33–2.37 percentage points for Qwen3-4B and 1.23–1.71 points for Qwen3-8B.

E.3 Planner Checkpoint Storage

Table 6 measures checkpoint storage in a separate set of KL-planned Qwen3-8B NVFP4 exports at the same three budgets. The reference is an existing plain NVFP4 checkpoint. Sizes include all files within each checkpoint directory.

The checkpoints exported at the three budgets occupy 3.55–9.53% more storage than plain NVFP4. Layers with partial compensation store copied weight rows, while full compensation reuses the base weight.

Table 5. Reasoning scores with AWQ (%). ResServe combines AWQ with linear residue compensation; BF16* is a reference. Best quantized scores per model and task are bold.

Method	Budget	LCB	MATH500	MMLU-Pro Math
Qwen3-4B				
BF16*	—	71.94	95.20	89.93
Plain NVFP4	—	61.42	**94.60**	87.64
AWQ	—	64.55	94.20	87.86
ResServe	4.5	65.88	94.00	**89.42**
ResServe	5.0	66.16	94.40	88.75
ResServe	5.5	**66.92**	94.40	89.34
Qwen3-8B				
BF16*	—	73.36	94.60	92.38
Plain NVFP4	—	65.59	95.60	90.97
AWQ	—	69.95	96.00	91.04
ResServe	4.5	71.37	95.00	91.41
ResServe	5.0	**71.66**	**96.20**	91.12
ResServe	5.5	71.18	96.00	**91.56**

Table 6. Qwen3-8B checkpoint storage. Budgets are planner effective bits, not physical bits per weight.

Configuration	Size (GiB)	Increase
Plain NVFP4	5.973	—
Budget 4.5	6.185	3.55%
Budget 5.0	6.428	7.62%
Budget 5.5	6.542	9.53%

Table 7. System quality with W4A8 baselines (%). LCB uses Hard350 pass@1; HMMT/AIME use avg@16. BF16 references use BF16 KV; all other rows use NVFP4 KV. ResServe prefill ratios are 0 for NVFP4 and 0.25 for MXFP4. Bold marks the highest score within each model and FP4 format.

Configuration		LCB	HMMT	AIME
Qwen3-4B	BF16	36.57	51.25	63.13
NVFP4	Plain	23.71	31.88	50.00
	W4A8	23.43	33.75	48.75
	ResServe	**28.86**	**37.92**	**58.75**
MXFP4	Plain	10.57	14.58	28.13
	W4A8	19.43	29.17	48.33
	ResServe	**20.86**	**33.96**	**50.83**
Qwen3-8B	BF16	45.14	55.83	68.33
NVFP4	Plain	24.29	43.13	56.25
	W4A8	30.86	45.63	60.21
	ResServe	**35.14**	**48.96**	**61.04**
MXFP4	Plain	12.00	22.08	36.88
	W4A8	21.43	38.33	51.88
	ResServe	**27.14**	**40.63**	**55.21**
Qwen3-32B	BF16	51.14	64.38	75.63
NVFP4	Plain	42.29	55.21	**71.46**
	W4A8	**46.57**	58.75	70.42
	ResServe	46.29	**61.67**	71.25
MXFP4	Plain	31.43	45.83	57.71
	W4A8	**40.29**	**55.00**	64.58
	ResServe	38.00	52.50	**67.08**
Qwen3.8-27B	BF16	84.00	79.38	84.17
NVFP4	Plain	79.43	**80.83**	**88.75**
	W4A8	**81.71**	77.50	83.96
	ResServe	77.71	78.96	88.33
MXFP4	Plain	70.86	66.25	72.92
	W4A8	**77.71**	72.08	**79.79**
	ResServe	74.86	**72.29**	78.75
Swift-27B	BF16	83.43	76.25	82.08
NVFP4	Plain	76.57	75.42	81.46
	W4A8	**80.57**	71.25	79.38
	ResServe	77.71	**78.33**	**85.42**
MXFP4	Plain	65.71	62.50	67.71
	W4A8	75.43	66.67	74.17
	ResServe	**77.43**	**69.17**	**75.83**

Table 8. Complete checkpoint storage with uniform compensation ratios. Sizes and additions are in GiB; increases are relative to same-format plain checkpoints and computed from unrounded byte counts.

Model	Plain	Size	Added	Increase
NVFP4, ratio 0.125				
Qwen3-4B	2.643	2.884	0.240	9.10%
Qwen3-8B	5.973	6.431	0.458	7.67%
Qwen3-32B	19.262	21.316	2.053	10.66%
Qwen3.8-27B	19.201	20.801	1.601	8.34%
Swift-27B	19.201	20.801	1.601	8.34%
NVFP4, ratio 0.25				
Qwen3-4B	2.643	3.123	0.480	18.16%
Qwen3-8B	5.973	6.888	0.915	15.32%
Qwen3-32B	19.262	23.365	4.103	21.30%
Qwen3.8-27B	19.201	22.399	3.198	16.66%
Swift-27B	19.201	22.399	3.198	16.66%
MXFP4, ratio 0.125				
Qwen3-4B	2.537	2.797	0.259	10.22%
Qwen3-8B	5.770	6.204	0.433	7.51%
Qwen3-32B	18.354	20.294	1.940	10.57%
Qwen3.8-27B	18.492	20.005	1.512	8.18%
Swift-27B	18.492	20.005	1.512	8.18%
MXFP4, ratio 0.25				
Qwen3-4B	2.537	2.991	0.454	17.87%
Qwen3-8B	5.770	6.635	0.865	14.99%
Qwen3-32B	18.354	22.230	3.876	21.12%
Qwen3.8-27B	18.492	21.514	3.021	16.34%
Swift-27B	18.492	21.514	3.021	16.34%

Table 9. NVFP4 HTTP latency (ms) across selective prefill ratios under the protocol in Appendix C.5. Values are means across three independent server processes.

Method	Ratio	TTFT	TPOT	E2E latency
Qwen3-8B				
W4A4	—	90.90	2.940	840.48
ResServe	0	91.58	3.048	868.76
ResServe	0.125	96.56	3.035	870.45
ResServe	0.25	100.82	3.061	881.35
Qwen3-32B				
W4A4	—	274.27	8.520	2446.96
ResServe	0	273.96	8.521	2446.91
ResServe	0.125	300.80	8.868	2562.15
ResServe	0.25	321.11	8.983	2611.87

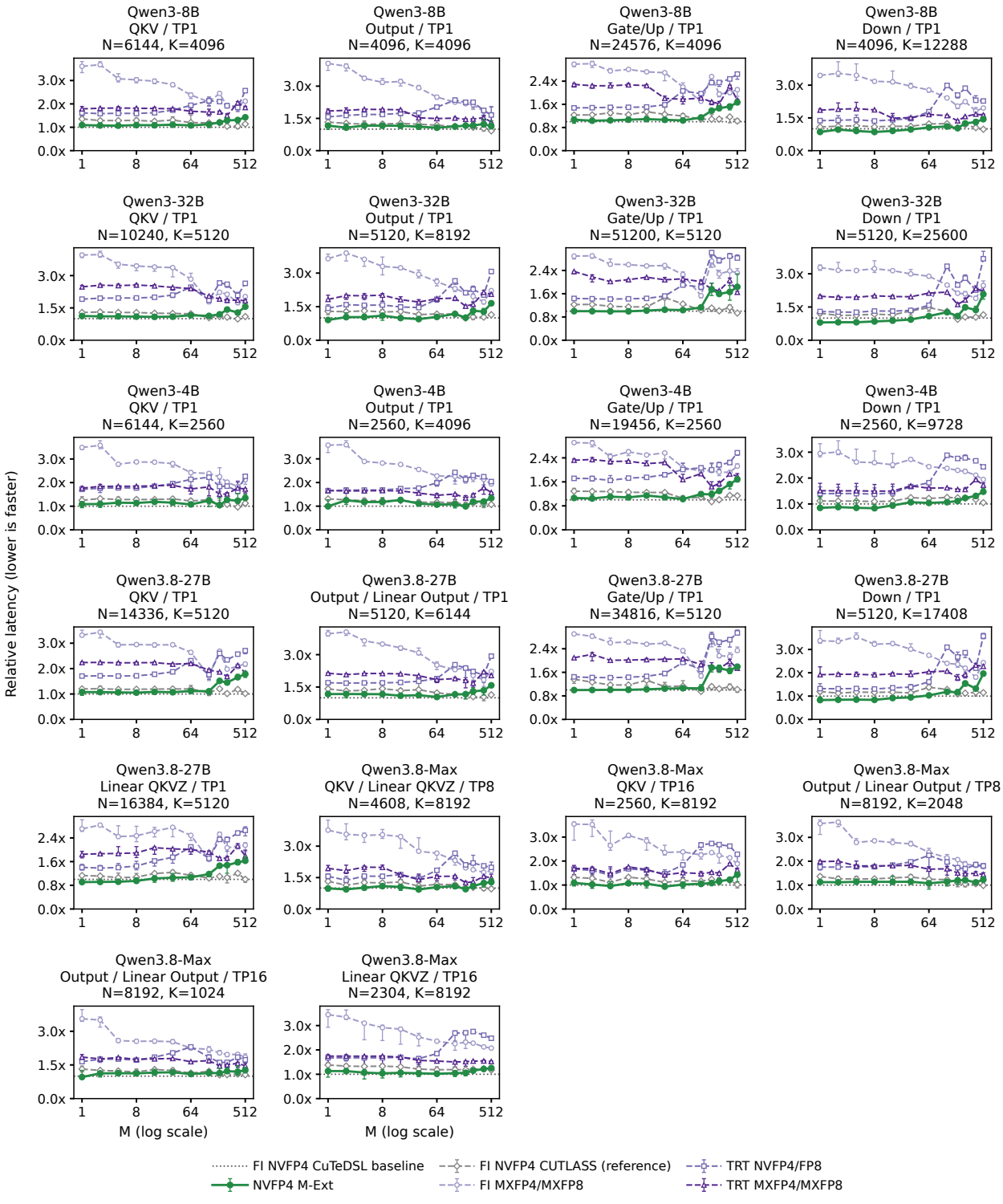

Figure 18. B200 NVFP4 M-Ext latency across 22 shapes, relative to plain FI NVFP4 CuTeDSL (lower is better).

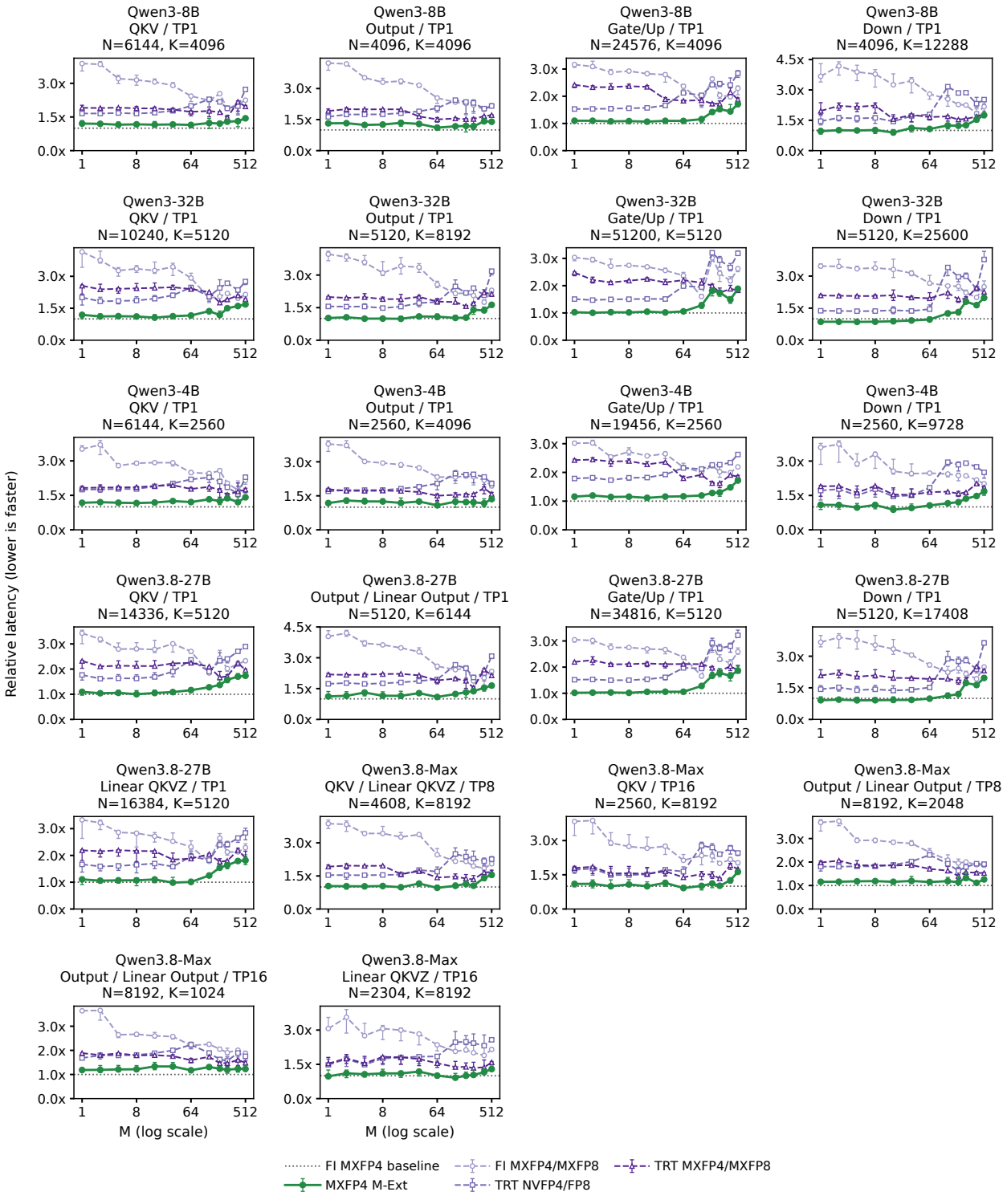

Figure 19. B200 MXFP4 M-Ext latency across 22 shapes, relative to plain FI MXFP4 (lower is better).

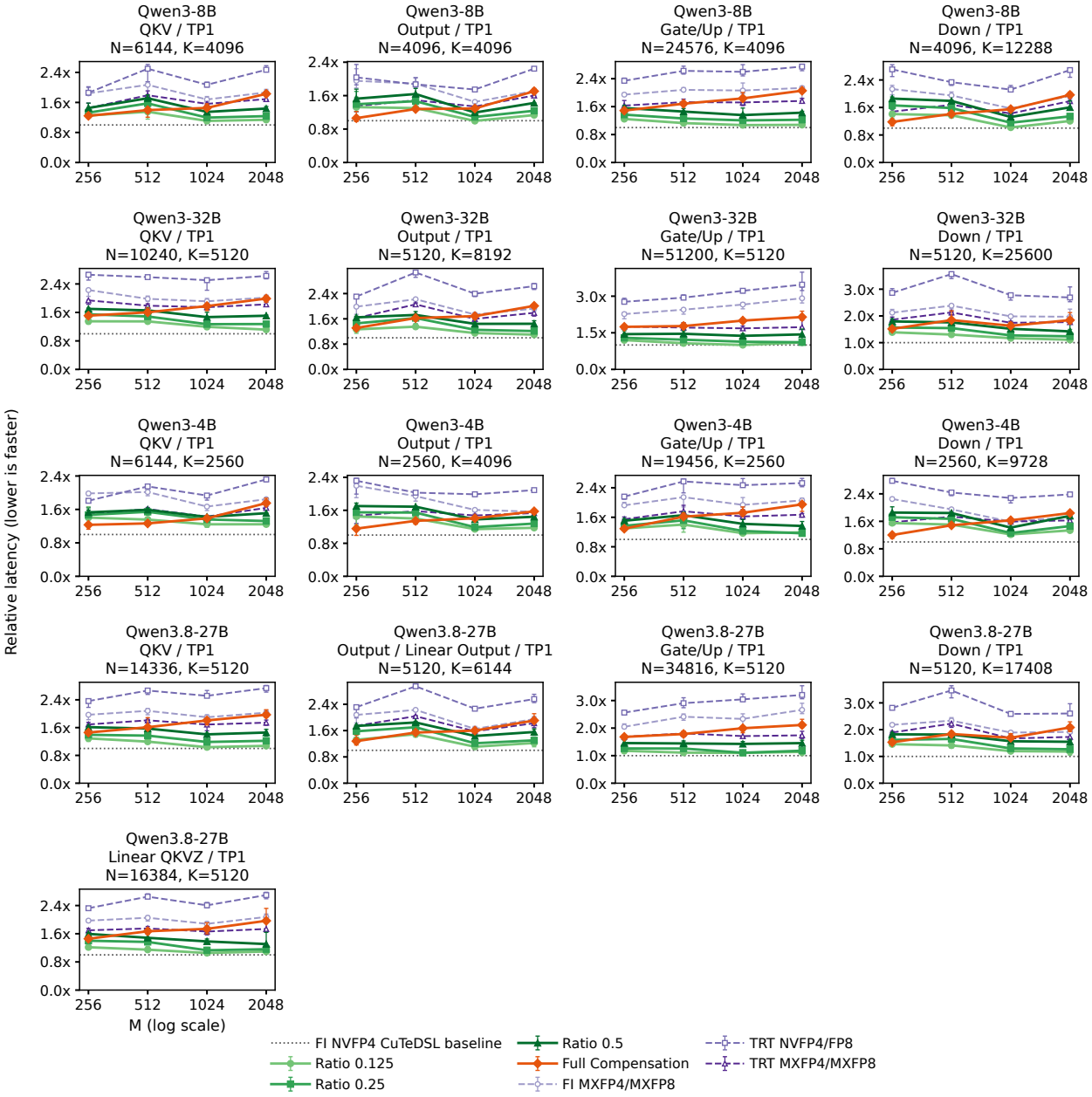

Figure 20. B200 selective prefill latency across 17 shapes, relative to plain FI NVFP4 CuTeDSL (lower is better).